

ALFOMs and the MOIRAI: Quantifying the Performance/Security Tradeoff for ZK-friendly Hash Functions

Aurélien Boeuf[✉] and Léo Perrin[✉]
aurelien.boeuf@inria.fr, leo.perrin@inria.fr

Inria, Paris, France

Abstract. Zero-Knowledge (ZK) protocols rely internally on hash functions for their security arguments. However, the hash functions that are the most efficient in this context differ substantially from e.g. SHA-3: their round function R must enable an efficient arithmetization of its verification. In practice, it means that verifying if $y = R(x)$ involves as little finite field multiplications as possible. In turn, this design requirement implies a greater vulnerability to algebraic attacks. In fact, improvement of those have proved devastating, and imply the need to completely rethink the methods used to ensure security against them.

In this paper, we show that it is possible to build a simple yet efficient security argument based on a precise estimate of the so-called “ideal degree” of a system of equations. Furthermore, we show that the increase of this quantity across rounds is tightly connected to the cost of the hash function in two different arithmetizations, namely AIR and R1CS. We precisely quantify this relation by introducing ALgebraic Figures Of Merit (ALFOMs) that capture how efficient a specific primitive (and in fact its round function) are at increasing the security per unit of cost.

This new insight allows us to better understand sometimes puzzling performance differences between state-of-the-art hash functions in the R1CS and AIR cases, and to provide a fair and simple comparison of their round functions in this context. Furthermore, we present a new group of round functions we called the MOIRAI which allow us to explore what a round function providing optimal performance/security tradeoff could look like.

Keywords: Arithmetization-Oriented · Zero-Knowledge · Gröbner Bases

1 Introduction

Recently, there has been a surge of modern advanced protocols based on Zero-Knowledge (ZK), Multi-Party Computation (MPC) and Fully-Homomorphic Encryption (FHE) techniques. Such protocols often require basic symmetric primitives, such as block ciphers and hash functions. However, standard primitives such as AES and SHA-3 were not designed with these applications in mind, and they do not integrate well within the frameworks of these advanced protocols:

they are defined over small binary fields and, while efficient to compute, their circuits have a complex arithmetic representation. By contrast, many cryptographers have recently proposed so-called Arithmetization-Oriented Primitives (AOP), that are defined over specific rings and odd-characteristic finite fields, or have simple circuit representations within specific frameworks. Indeed, a key step in the evaluation of these primitives within the various protocols is their “arithmetization”, a process whereby an arithmetic circuit is derived from the specification of the primitive. Each protocol will allow different operations within said circuits, as well as imposing a different cost for different gates.

In this paper, we narrow our focus specifically to ZK-friendly families of hash functions. This subset of AOP has received a lot of attention in recent years, both in terms of design and in terms of cryptanalysis. Prominent examples include Rescue [2], Rescue-Prime [32], RPO [4], Anemoi [12], Griffin [23] and Poseidon [24].¹ They are all designed by combining affine operations with “simple” non-linear operations, such as power maps.

Design Requirements. Designing a ZK-friendly hash function has meant, in practice, designing a permutation which would then be used to construct a sponge, or sometimes a compression function. In turn, said permutation consists in multiple iterations of a simple round function obtained by composing an affine layer with a non-layer. While at this point nothing has come out of the ordinary of hash function design, the following additional requirements are unusual:

1. the arithmetization will be done over a specific field imposed by the protocol, typically the scalar field of some pairing-friendly elliptic curve, or the modular ring corresponding to the so-called *Goldilocks prime* $p = 2^{64} - 2^{32} + 1$, meaning that the round function should be defined over a vector space of the same field,
2. the round function R must be such that it is “easy” to verify that $R(x) = y$ given both x and y .

What is “easy” depends on the specific of the protocol and its underlying arithmetization, but a rule of thumb is that it should involve only low degree operations over the field. However, this approximation has its limits, as we will show.

R1CS and AIR. The three main families of arithmetization for ZK protocols are the variants of Plonk, R1CS and AIR. In this paper, we will leave aside Plonk as it allows more complicated operations which increases the impact of the implementer’s work on the final performance. On the other hand, the efficiency metrics for both R1CS and AIR are rigid enough that we will be able to make strong statements about the suitability (and in fact, the optimality) of some

¹ We purposefully leave aside hash functions based on the “Split-and-lookup” or “Kintsugi” approach, like Monolith. These hash functions shine in the arithmetization by some variants of the Plonk ZK protocol, which differs significantly from both AIR and R1CS on which we will later focus our attention.

hash functions and their round functions for these arithmetizations. They are also somewhat similar in that the designers’ goal for both is indeed a priori to limit the number of multiplications. While we leave a description of the details of these metrics for later (Section 3), we can already make some perhaps puzzling observations about the state of the art. Consider for instance Table 1, which is essentially lifted from [12]. It describes the cost of various ZK-friendly hash functions for different arithmetizations, and for different field size properties.

Condition	Proof system	Rescue-Prime	Poseidon	Griffin	Anemoi
$p - 1 \not\equiv 0 \pmod{3}$	R1CS	224	232	112	96
	AIR	168	384	168	168
$p - 1 \not\equiv 0 \pmod{5}$	R1CS	264	264	110	120
	AIR	220	440	220	280

Table 1: Performance of various ZK-friendly hash operating on $\mathbb{F}_p^4$ in different proof systems (taken from Table 2 of [12]). Smaller numbers are better.

As we can see, there are some common tendencies. If the field size prevents $x \mapsto x^3$ from being bijective, in which case we are in the bottom rows, then the cost is higher, sometimes much higher. Similarly, Poseidon is never competitive. On the other hand, the Griffin/Anemoi pair on one side and Rescue-Prime on the other have opposite properties regarding AIR and R1CS: the first are by far the best for R1CS, but they completely lose their edge for AIR. In fact, both have been the target of successful full-round attacks, meaning secure instances (using more rounds) would lose out to Rescue-Prime. In summary, this cursory study shows several things.

1. R1CS and AIR are similar enough that general tendencies across both exist.
2. On the other hand, detailed performance metric can still vary significantly.
3. Comparing AOP is made harder by the very quick advances in their cryptanalysis which has already made obsolete several instances that should have their round number increased.

Our goal in this paper is to solve these problems by providing a metric for each arithmetization which combines information about their efficiency *and their security*. However, in order to discuss the latter, we first recall how the cryptanalysis of AOPs works.

Polynomial System Solving. From the inception of this field of study, it has been clear that an important threat to these primitives was *Polynomial System Solving* (PoSSo). Such techniques are used to find the solution(s) of a system of multivariate equations. As mentioned above, the round functions of AOPs must be such that verifying if $y = R(x)$ involves only the evaluation of low degree polynomials. As a consequence, for any attack, there should exist a system of low

degree multi-variate equations whose solution is what the cryptanalyst wants, such as a preimage.

As a consequence, the first iterations of Rescue were already designed in a way intended to ensure that PoSSO algorithms would have a prohibitive complexity when used to attack them. Since then, all AOP designers have provided arguments to the same effect.

In practice, PoSSO is done using the following steps.

Equation writing. Rewrite the intended cryptanalysis as a system of multivariate polynomial equations.

Obtain a univariate polynomial. If the equations are multivariate, derive a univariate equation from them. The methods for this always involve either resultants or Gröbner bases, and are summarized as follows:

1. Compute a Gröbner basis of the multivariate system for some monomial ordering using a standard algorithm such as F4 or F5.
2. Deduce a new Gröbner basis for the monomial ordering “lex”, or use the eigenvalue method to find a univariate polynomial.

Final resolution. Find the roots of said univariate polynomial, and substitute them with the corresponding variable to find the values of the remaining variables (possibly by solving a strictly simpler multivariate system).

Devastating Attacks. The security of most AOPs against such attacks has been argued by estimating the cost of the Gröbner basis computation, and ensuring that the number of rounds was high enough to make it infeasible. Unfortunately, several recent breakthroughs have shown this approach to be insufficient. Resultant-based attacks [33,6] and Gröbner basis attacks, such as the Freelunch attacks [7] or a dedicated approach to compute Gröbner bases efficiently [15] turned out to break full-round instances of Griffin, Anemoi, Arion and Rescue, far past the analysis of their designers. We identify two main reasons for this:

1. The designers of these primitives all relied on the hardness of computing Gröbner bases of the polynomial models to compute their round numbers.
2. Computing a Gröbner basis of these models is in some cases cheap (or even free in the case of the Freelunch attacks) or not required altogether in the case of resultant-based attacks.

A better approach is thus needed to ensure the security against such attacks.

On the Importance of the “Ideal Degree”. All of these attacks still involve on a certain quantity that is a fundamental attribute of the polynomial model: the degree of the ideal spanned by the multivariate polynomials, denoted D_I . The complexity of the attacks described above are at least linear in D_I , and at most cubic in it. Furthermore, and crucially, D_I does not depend on the intricacies of how the system is obtained, it is in fact an inherent property of the attack being modelled. As such, using a different method to derive the equations will not modify it.

The only techniques known to reduce D_I are the so-called “round skips”, which, as their name implies, leverage particular structures at the input or output to avoid needing to define a few variables or equations within the first few rounds, effectively skipping (part of) them.

We therefore argue that convincing security arguments against PoSSo-based attacks can be made by lower-bounding the ideal degree corresponding to various attacks.

Our contributions. The key insight in this work is a surprising relationship between D_I and the performance of an AOP in either R1CS or AIR. Indeed, when comparing the growth of D_I with the number of rounds in state-of-the-art PoSSo cryptanalysis, and the increase in cost of the corresponding primitives within either R1CS or AIR with the number of rounds, clear, simple relations emerge. We call these *ALgebraic Figures Of Merit* (ALFOMs).

In this paper, we formally define ALFOMs for AIR and R1CS, and show that are relevant yet simple metrics. They allow us to evaluate the design of existing ZK-friendly hash functions in a unified, coherent way by capturing the performance/security trade-off of their round function. This approach solves all the problems we identified above.

- The security level of AOPs is easily computed using this method, meaning that it is easy to update the number of rounds of broken primitives to provide a given security level. A comparison at an equivalent security level then becomes possible.
- The similarities and differences between R1CS and AIR are captured by their differing ALFOMs. In fact, we show that the ALFOM of Rescue-Prime for R1CS is much worse than that of Anemoi or Poseidon, while the exact opposite is true for AIR: the discrepancies in Table 1 are directly explained using this tool.

Beyond a better understanding of the state-of-the-art, we also show that ALFOMs can serve as guiding principles when building round functions. In fact, we can even argue that a round function is *optimal*, in the sense that it allows the best possible increase of the idea degree for a given AIR or R1CS cost. We showcase this possibility by introducing a new group of ZK-friendly hash functions we call the MOIRAI: MOIRAI: CLOTHO, LACHESIS and ATROPOS. These constructions allow us to explore the limits of maximizing ALFOMs across both R1CS and AIR.

Outline. In Section 2, we recall the definition of some ZK-friendly families of permutations used to design hash function, their cost models and state-of-the-art cryptanalysis methods used against them. In Section 3, we introduce the ALFOMs metrics, justify their definition and relevance, and we give some optimal bounds on them. In Section 4, we formally evaluate the ALFOMs of Rescue, Poseidon and Anemoi, and comment their designs in the light of their ALFOMs. Lastly, in section 5, we introduce the MOIRAI constructions that were conceived to maximize ALFOMs (and reach optimal values) upon various constraints.

2 ZK-friendly hash functions and their analysis

2.1 Some ZK-friendly hash functions

In this section, we describe the families of permutations Anemoi [12], Rescue-Prime [32] and Poseidon [24], which we will study in more detail in later sections. All of these are used to instantiate hash functions using the sponge construction.

Poseidon. Poseidon is a family of ZK-friendly permutations based on the simple $x \mapsto x^\alpha$ permutations (for α coprime to $p-1$). In this paper, we will mainly consider Poseidon [24], rather than Poseidon2 [25], the latest iteration, because more detailed cryptanalysis exists for it, though we expect Poseidon2 to have a very similar analysis. Poseidon is defined over $\mathbb{F}_p^b$ for $b \geq 2$. One permutation consists of R_F “full” rounds and R_P “partial” rounds: a full round’s nonlinear layer applies the $x \mapsto x^\alpha$ permutation on every branch, while a partial round only applies it on one branch. A figure of the Poseidon permutation is given in Figure 3. Mathematically, it has the following shape:

$$\underbrace{\mathcal{M} \circ \mathcal{S}_F \circ ARC_{R_F+R_P} \circ \dots \circ \mathcal{M} \circ \mathcal{S}_P \circ ARC_j \circ \dots \circ \mathcal{M} \circ \mathcal{S}_F \circ ARC_1}_{R_F/2 \text{ rounds}}, \quad \underbrace{\phantom{\mathcal{M} \circ \mathcal{S}_F \circ ARC_{R_F+R_P} \circ \dots \circ \mathcal{M} \circ \mathcal{S}_P \circ ARC_j \circ \dots \circ \mathcal{M} \circ \mathcal{S}_F \circ ARC_1}}_{R_P \text{ rounds}}, \quad \underbrace{\phantom{\mathcal{M} \circ \mathcal{S}_F \circ ARC_{R_F+R_P} \circ \dots \circ \mathcal{M} \circ \mathcal{S}_P \circ ARC_j \circ \dots \circ \mathcal{M} \circ \mathcal{S}_F \circ ARC_1}}_{R_F/2 \text{ rounds}},$$

where $\mathcal{S}_F$ is the application of $x \mapsto x^\alpha$ on every branch, $\mathcal{S}_P$ is its application on only one branch, $\mathcal{M}$ denotes multiplication by a given matrix and ARC_j is the round constant addition of the j -th round.

Rescue. For this paper we focus on Rescue-Prime [32], as it is the most studied member of the Rescue family. Rescue-Prime features the power maps $x \mapsto x^\alpha$ and $x \mapsto x^{1/\alpha}$ as its Sboxes, where $\alpha = \min\{\alpha' : \gcd(\alpha', p-1) = 1\}$. The round function of Rescue-Prime is given in Figure 4. The full permutation is described as

$$\mathcal{A}_{n_r,1} \circ \mathcal{S}^{-1} \circ \mathcal{A}_{n_r,0} \circ \mathcal{S} \circ \dots \circ \mathcal{A}_{1,1} \circ \mathcal{S}^{-1} \circ \mathcal{A}_{1,0} \circ \mathcal{S}$$

where $\mathcal{S}$ denotes the application of $x \mapsto x^\alpha$ on every branch, and $\mathcal{A}_{j,0}$ and $\mathcal{A}_{j,1}$ are the two affine layers of the j -th round, $1 \leq j \leq n_r$.

Anemoi. Anemoi [12] is a family of permutations that has two main variants: one in even characteristic and one in odd characteristic. We will mainly study the odd characteristic case, as it is the only one for which the original authors gave benchmarks. It is also the most frequently investigated case. The number of branches b is always taken as an even integer (due to the Feistel-like structure of the nonlinear layer). Anemoi can be decomposed as follows:

$$\mathcal{A}_{n_r} \circ \mathcal{S} \circ \dots \circ \mathcal{A}_1 \circ \mathcal{S} \circ \mathcal{A}_0,$$

where the $\mathcal{A}_j$ are affine layers (their full description is a bit complicated, we refer to the original paper) and $\mathcal{S}$ denotes the successive application of the following maps (a so-called “Flystel”) on each pair of branches:

$$((x, y) \mapsto (x + gy^2, y)) \circ ((x, y) \mapsto (x, y - x^{1/\alpha})) \circ ((x, y) \mapsto (x - gy^2 - g^{-1}))$$

where g is a given element of $\mathbb{F}_p$. Figure 5 represents the first few layers.

2.2 On ZK Constraint Systems and their Cost Functions

In this section, we give an overview of the Zero-Knowledge constraint systems R1CS and AIR, and we give a reminder on how designers have been able to evaluate how efficient primitives are relative to each of these constraint systems through cost functions.

R1CS. In the context of Rank One Constraint Systems (R1CS) [31], the permutation is modeled as a multivariate polynomial system where each equation has one of these forms:

$$\begin{cases} z = x + y & \text{where } z \text{ is a variable and } x, y \text{ are variables or constants,} \\ z = x \cdot c & \text{where } x, z \text{ are variables and } c \text{ is a constant, or} \\ z = x \times y & \text{where } x, y, z \text{ are variables.} \end{cases}$$

Such a polynomial system is then translated as a constraint system in the context of a Zero-Knowledge protocol. Out of these three types of equations, the variable-variable multiplications end up being, by far, the most costly ones in the context of the protocol. Therefore, a permutation that is efficient within the context of R1CS is a permutation that can be modeled with a “small” number of multiplications.

Note that a primitive of low-degree, such as MiMC [1] or Poseidon [24], will automatically yield low-degree models. However, it is perfectly possible to mix operations which either have low degree or have an inverse of low-degree. This strategy is prominently used in [32], where the Sboxes $x \mapsto x^\alpha$ and its inverse (for a low value of α , such as 3, 5 or 7) are used.

Definition 1 (R1CS Cost). *Let P be a permutation. The R1CS Cost of P is denoted $C_{\text{R1CS}}(P)$ and corresponds to the number of multiplications in a given R1CS model of P .*

Example 1 (Rescue-Prime). In Rescue-Prime [32], the number of Sboxes is $2 \cdot n_r \cdot b$, where b is the number of branches and n_r is the number of rounds. Consider a Rescue-Prime permutation P for any fixed p , n_r and b and define α as the smallest integer such that $x \mapsto x^\alpha$ is a permutation. Each Sbox is either $x \mapsto x^\alpha$ or its functional inverse $x \mapsto x^{1/\alpha}$, where $1/\alpha$ is the multiplicative inverse of α modulo $p-1$. The authors of Rescue-Prime compute the R1CS Cost as follows: each Sbox can be evaluated with a certain number of multiplications which depends on α (see Section B), which we write as $\ell(\alpha)$. Then, the R1CS Cost is

$$C_{\text{R1CS}}(P) = 2 \cdot \ell(\alpha) \cdot n_r \cdot b.$$

AIR. The Algebraic Intermediate Representation (AIR) is a kind of arithmetization that was conceived for the STARK protocol, along with primitives like Rescue-Prime. It consists of the following elements:

1. A set of $t \cdot w$ variables $\{x_{i,j}\}$ where t is the number of steps of the evaluation, and w is the “width”, the number of registers that we specify at each step (typically, the values taken by the branches of a block cipher after i rounds).
2. A secret matrix of size $t \cdot k$, called the “execution trace”, where the coefficient i, j is the value taken by $x_{i,j}$ (the j -th register at step i).
3. A public set of polynomial constraints on variables $\{x_{i,j}\}$.

Definition 2 (AIR Cost). Let P be a permutation, and consider an AIR model of P with $t \cdot w$ variables such that $d_{\max}$ is the maximum total degree of the polynomial constraints. Following [2], the AIR Cost of P is $C_{\text{AIR}}(P) = t \cdot w \cdot d_{\max}$.

Example 2 (Rescue-Prime AIR). For Rescue-Prime, $x_{i,j}$ is essentially the value of the j -th variable after i rounds. The specifics may vary, but for b branches and r rounds, we typically get a set of $r \cdot b$ polynomial constraints of total degree α , where $\alpha = \min\{\alpha' : \gcd(p-1, \alpha') = 1\}$. Then, the AIR Cost is simply $r \cdot b \cdot \alpha$.

2.3 Attacking a Hash Function using PoSSo

In the absence of a “secret key”, the security of public permutation families used in sponge constructions [32,4,23,12,29] is often formulated in terms of the hardness of the CICO problem.

Definition 3 (CICO problem). Let $\mathbb{F}$ be a field and n, k be positive integers. Given a permutation $\mathbf{P} : \mathbb{F}^n \rightarrow \mathbb{F}^n$, we define the CICO- k problem as follows: find $\mathbf{x} \in \mathbb{F}^n$ such that both $\mathbf{x} \in \mathbb{F}^{n-k} \times \{0\}^k$, and $\mathbf{P}(\mathbf{x}) \in \mathbb{F}^{n-k} \times \{0\}^k$.

A brute-force strategy to solve CICO- k for $\mathbf{P}$ consists in evaluating $\mathbf{P}$ on random inputs $\mathbf{x} \in \mathbb{F}^{n-k} \times \{0\}^k$ until we find an output $\mathbf{P}(\mathbf{x}) \in \mathbb{F}^{n-k} \times \{0\}^k$. If $\mathbb{F}$ has size q and if $\mathbf{P}$ is a random permutation, then we can expect this approach to require q^k evaluations of $\mathbf{P}$ on average. On the other hand, a common approach to find solutions to this problem in a more efficient way consists in reducing CICO problems to PoSSo problems. Such techniques fall under the (very broad) umbrella of *algebraic cryptanalysis*. In the following sections, we describe the main CICO-to-PoSSo reductions currently used in cryptanalysis.

Univariate Root-Finding. Univariate root-finding algorithms can be used to solve the CICO-1 problem. It performs better than brute-force under the condition that the permutation considered has a “low enough degree”. The expected number of solutions is easily established using the results in [21], and summarized in Proposition 1.

Definition 4 (CICO-1 to Univ). Let $\mathbf{P} : \mathbb{F}^n \rightarrow \mathbb{F}^n$ be a permutation. Consider $\mathbf{P}$ restrained to any affine space A of dimension 1 parametrized by a single variable x in $\mathbb{F}_q$, that verifies the following equation:

$$A = \{x\mathbf{v} + \mathbf{a} : x \in \mathbb{F}_q\} \subset \mathbb{F}_q^{n-1} \times \{0\}.$$

Let $\mathbf{P}_{\mathbf{v},\mathbf{a}} : \mathbb{F}_q \rightarrow \mathbb{F}_q$ be the function that maps $x\mathbf{v} + \mathbf{a}$ to the n -th coordinate of its image by $\mathbf{P}$. Then, a root of this function is a solution to the CICO-1 problem.

Proposition 1 (Probability of finding a root). The probability that a random function $f : \mathbb{F}_q \rightarrow \mathbb{F}_q$ has a root is $\Pr_f[\exists x \in \mathbb{F}_q : f(x) = 0] = 1 - (1 - 1/q)^q$, which converges to $1 - 1/e \approx 0.63$ as $q \rightarrow +\infty$. More generally, the number of roots of a random function follows a Poisson law of parameter 1.

In order to find the roots of the given univariate polynomials, rather than directly applying the general Berlekamp [10] or Cantor-Zassenhaus [16] algorithms, one can first compute their greatest common divisor with the field polynomial $x^q - x$ using a fast half-GCD [30] algorithm. This will yield a polynomial whose degree follows a Poisson distribution of parameter 1 (see Proposition 1). That polynomial has average degree 1, and can be factorized in a negligible time (compared to the time it takes to compute the gcd). This method is used in state-of-the-art algebraic cryptanalysis [8,7]. Thus, the complexity of computing the roots of a random polynomial of degree d in $\mathbb{F}_q$ is

$$\mathcal{O}(d \log(d) (\log(d) + \log(q)) \log(\log(d))) . \quad (1)$$

Efficient univariate root-finding is frequently used in various PoSSo algorithms, even in the multivariate case. The problem reduction from Definition 4 is particularly relevant for the cryptanalysis of arithmetization-oriented primitives whose round function has low degree, such as MiMC [1] or Poseidon [24,25].

Remark 1 (CICO- k to CICO-1 reduction). Suppose that you can solve the CICO-1 problem faster than brute-force using the reduction from Definition 4 for approximately q^{k-1} distinct affine spaces A such that each of these affine spaces is included in $\mathbb{F}_q^{n-k} \times \{0\}^k$. Then, it is possible to solve the CICO- k by computing a solution to the CICO-1 problem where the input is in $\mathbb{F}_q^{n-k} \times \{0\}^k$. Then, there is a probability of $1/q^{k-1}$ that the $k-1$ “leftover” coordinates are all equal to 0, which would make this CICO-1 solution a CICO- k solution.

Gröbner Basis Solving. For ZK-friendly hash functions, it is the *verification* that needs to correspond to low degree polynomials, i.e. the operation taking the (x, y) as input that returns true if $y = R(x)$. Thus, designers (starting with those of Rescue [2]) have relied not only on low degree functions, but also on their functional inverse. This trick means that the permutation quickly reaches a very high degree, rendering the above approach useless. However, it is then possible to introduce intermediate variables and new equations to obtain a multivariate system only containing low degree equations.

Definition 5 (CICO- k to Gröbner Basis Solving). Let $\mathbf{P} : \mathbb{F}^n \rightarrow \mathbb{F}^n$ be a permutation and $k < n$. Consider $\mathbf{P}$ restrained to any affine space A of dimension k parametrized by k variables $x_1, \dots, x_k$ in $\mathbb{F}_q$, that verifies the following equation for some vectors $\mathbf{v}_i$ and $\mathbf{a}$:

$$A = \left\{ \sum_{i=1}^k x_i \mathbf{v}_i + \mathbf{a} : x_1, \dots, x_k \in \mathbb{F}_q \right\} \subset \mathbb{F}_q^{n-k} \times \{0\}^k.$$

Suppose that we are able to introduce intermediate variables $y_1, \dots, y_r$ such that there exists a polynomial system

$$\begin{cases} P_1(x_1, \dots, x_k, y_1, \dots, y_r) = 0, \\ \vdots \\ P_m(x_1, \dots, x_k, y_1, \dots, y_r) = 0, \end{cases}$$

whose solutions include CICO- k solutions (this constraint dictates the definition of the polynomials). Then, the solutions of this system are CICO- k solutions.

Complexity of Gröbner Basis Solving. Consider a polynomial system like the one in Definition 5, and suppose that the ideal spanned by the P_i for $1 \leq i \leq m$ has dimension 0.² Then, solving that system is done in the three main steps, as described in the introduction: the first two are intended to find a univariate polynomial, while the last is a univariate root-finding step. The first step consists in computing a Gröbner Basis of the system for any monomial order using an algorithm such as F4 [17] or F5 [18], and the second one consists in changing this monomial order to *lex* using an algorithm such as FGLM [19]. These first two steps are the bottleneck in practice as the last step can be done in quasi-linear time (see Section 2.3).

Some authors [12,32] explicitly try to evaluate the time complexity of finding a Gröbner basis using the F4 or F5 algorithms, and use this to determine the number of rounds needed for a given security level. However, it has been recently shown [7,15] that, in many cases, it can be much easier (or even “free”) to find a Gröbner Basis in a specific monomial ordering. In this case, the second step becomes the bottleneck. Its complexity is formulated as a function of the degree of the ideal, D_I , a very important quantity which we will discuss in Section 3.1 in more detail.

Proposition 2 (FGLM complexity). Let I be an ideal of dimension 0 of $\mathbb{F}[x_1, \dots, x_n]$ and G be a Gröbner Basis for I in any monomial ordering $\prec$. Let D_I be its degree. Then the FGLM algorithm [19] computes a Gröbner Basis in lexicographical ordering with a time complexity of $\mathcal{O}(nD_I^3)$.

² This condition is needed by the algorithms described further. Its specific definition is too involved for this discussion, but is satisfied in our case as we expect the system to have 1 or a couple solutions.

3 Introducing ALFOMs

3.1 On the Importance of the Ideal Degree

In Section 2.3, we described a general approach to solve polynomial systems of equations. The FGLM algorithm is generic but, in some cases, there exist better ways to compute a univariate polynomial from a PoSSo instance, such as Sparse-FGLM variants [20], dedicated optimized algorithms [7], or even resultant-based methods [6,33]. All these methods can yield a univariate polynomial without the need of a Gröbner Basis. However, all of these methods depend at least quasi-linearly in the degree of the ideal, D_I . Note that under generic assumptions, D_I is also the smallest degree of all univariate polynomials in the ideal. As a consequence, we make the following claim, which will serve as the basis for the reminder of this paper.

Claim. The ideal degree is the most important quantity to study when evaluating the hardness of PoSSo-based cryptanalysis in Arithmetization-Oriented primitives, and a lower bound on its value is a sufficient security argument against PoSSo-based attack.

It is worth mentioning that the authors of XHash provided some security arguments based on conjectured lower bounds on D_I [3, Conjecture 1].

3.2 Quantifying the Performance/Security Tradeoff

This claim motivates our definition of a metric that defines the security-to-performance ratio of AOPs for a given arithmetization using:

- D_I as a representative of the “security” of a public permutation against PoSSo cryptanalysis, and
- a relevant cost function for the given arithmetization as a representative of the “performance”.

Moreover, we observe that in a lot of relevant works, D_I grows exponentially with the number of rounds, whereas the number of constraints in the underlying ZK-protocol grows linearly with the number of rounds. Thus, we can define a metric for the overall efficiency of a public ZK-friendly permutation that is invariant in the number of rounds that we call Arithmetic Efficiency.

Definition 6 (ALFOMs). Let $(P(n_r))_{n_r}$ be a family of public permutations defined for any number n_r of rounds. Fix an integer k and suppose that for any $n_r \geq 1$, we can derive a PoSSo instance of the CICO- k problem that spans an ideal of degree $D_I(n_r, k)$. Let the cost function in the underlying ZK protocol be $C(n_r)$. Then, we define the Algebraic Figures Of Merit (ALFOMs) of this family as follows:

$$\eta(P(n_r), k) = \frac{\log_2(D_I(n_r, k))}{C(n_r)}.$$

Furthermore, assume that there exist d_k and c such that for all $n_r \geq 1$, $D_I(n_r, k) = d_k^{n_r}$ and $C(n_r) = n_r c$. Then, $\eta(P(n_r), k)$ is constant in n_r , and we simply write $\eta(P, k) = \log_2(d_k)/c$.

If the ALFOMs does not depend on the number of rounds, then it is in fact a property of the round function of the primitive. This allows us to compare the actual efficiency of some ZK-friendly families of permutations simply by looking at their round functions. While this approach is broad and loses some nuance, such as round skips and how the number of rounds is defined, we will show in the rest of this section that it proves relevant in practice, and we will examine how it can guide design. In particular, in Section 3.3, we will define optimality bounds in the context of R1CS and AIR-related ZK systems of constraints, with the associated “optimal” designs according to this metric.

Choice of Polynomial System. In the previous definition, D_I is dependent on the polynomial systems chosen by the cryptanalyst (and in particular, depends on k). In the simplest cases, D_I will have the form $d_k^{n_r}$. However, the cryptanalyst has some leverage to decrease D_I : when solving a CICO- k problem, they can choose a subset of inputs which does not activate some nonlinear layers. This is what we call “round skips” in the context of PoSSo-based cryptanalysis. When modelling the action of P on such inputs, some equations can therefore be removed, or have lower degree, or fewer monomials, which can decrease D_I and the complexity of the overall attack. It is generally an open question to determine the models that achieve the smallest D_I .

Relation between ALFOMs and full cost. In the case of AOPs where the best cryptanalysis (aside from brute force) tends to come from PoSSo-based attacks, there is a direct relation between ALFOMs and the ZK-cost of the whole hash function. Indeed, the number of rounds is chosen so that the best known algorithms to solve CICO- k for a certain k (typically half the capacity of the sponge) are greater than a given security bound 2^λ . Fix a ZK system and consider the simple case where we can write $\eta(P, k) = \log_2(d_k)/c$. When the number of rounds is chosen³ so that $\log_2(D_I(n_r, k)) \approx \lambda$, we can write the cost function C as a function of λ and we have that $C(\lambda) \approx \lambda/\eta(P, k)$.

More formally, assuming that the number of rounds is computed as $n_r = \lceil \frac{\lambda}{\log_2(d_k)} \rceil + c$ where c is the number of bonus marginal rounds, we have that as λ increases,

$$C(\lambda) = \frac{\lambda}{\eta(P, k)} + o(1) .$$

3.3 Optimality bounds

ALFOMs cannot take arbitrary values: as we will show, there are upper bounds, meaning that there is such a thing as an *optimal* ZK-friendly hash function—at least in theory. In what follows, we derive upper bounds for ALFOMs for both R1CS and AIR. There are two main cases to consider. The first is the

³ Which is true for Poseidon instances where $k = 1$, but an “approximation error” appears since a few rounds are always added to account for round skips and to give some security margin.

“Poseidon-like” case, where all of the components of P (or P^{-1}) have low degree. In this case, to model a CICO- k instance, one typically only needs to introduce k variables at the input (resp. output). The second is the “Rescue-like” case, where there is an alternating use of low-degree and low-degree-inverse components, which forces the presence of many intermediate variables in the PoSSo instance.

R1CS ALFOMs Bound. Recall from Section 3 that, in the R1CS case, the ALFOMs is the ratio between $\log_2(D_I)$ and the number of multiplications in the underlying arithmetic circuit. Using m multiplications, the greatest power of x that can be achieved⁴ is x^{2^m} . In fact, we deduce from the classical Bézout bound that, given an R1CS system with m multiplications (and thus a polynomial system of equations), D_I can be no more than 2^m , no matter which CICO- k instance is being studied. Thus, we get the following proposition.

Proposition 3 (R1CS bound). *If a permutation P can be arithmetized in R1CS then, for any k , $\eta_{R1CS}(P, k) \leq 1$.*

Proof. Any dimension-0 model whose variables appear in the R1CS has at most as many solutions as the “naive” models where every atomic operation has an equation, and whose equations are therefore a subset of the equations that appear in the R1CS model. Furthermore, let m be the number of multiplications appearing in this model. Polynomial systems used to attack this primitive would have at most m equations of degree 2, while all the others have degree 1. By the Bézout bound, we have that $D_I \leq 2^m$. Thus, we deduce that $\log_2(D_I) \leq m$, and that $m = C_{R1CS}(P)$. $\square$

Optimal R1CS constructions. Given the existence of an upper bound, it is natural to ask whether it is tight. Are there optimal constructions in the sense that, to solve any CICO- k on any number of rounds, $\eta_{R1CS}(P(n_r, k)) = 1$? This question is not easy to answer due to the existence of the “round skips” mentioned in Remark 3.2. However, under the hypothesis that the decrease of D_I due to round skips is upper-bounded, it is possible to define constructions whose ALFOMs converges to 1 as the number of rounds increases using quadratic non-linear function (see Section 5). In fact, the importance of quadratic functions in this context is well known by designers, and they have been extensively studied [27, 22].

AIR ALFOMs Bounds Much like with R1CS, the existence of an efficient AIR modeling directly yields an upper bound on D_I . Recall that, for a permutation P , $C_{R1CS}(P) = w \cdot t \cdot d_{\max}$. With those notations, an AIR model consists of $w \cdot t$ equations of total degree at most $d_{\max}$ in variables that represent values taken by the elements of the state at different steps of the computation. From those, one can derive a dimension-0 ideal to model any CICO problem, where the Bézout bound gives $D_I \leq d_{\max}^{w \cdot t}$. We deduce the following proposition.

⁴ A more thorough discussion on this topic involves so-called *addition chains* and is provided in Section B.

Proposition 4 (AIR bound). *If a permutation P can be modeled by an AIR containing equations of degree at most $d_{\max}$, then, for any value of k ,*

$$\eta(P, k) \leq \frac{\log_2(d_{\max})}{d_{\max}}.$$

The maximum is reached for $d_{\max} = 3$, in which case $\eta(P, k) \leq \log_2(3)/3 \approx 0.53$.

4 Case Studies: the ALFOMs of some primitives

In this section, we give explicit values for the ALFOMs coefficient for the families of primitives that we presented in Section 2.1: Rescue, Poseidon, and Anemoi for both R1CS and AIR. The D_I values are taken from the best existing analysis and round skip methods. Most importantly, we do not just compute ALFOMs for fixed parameters: we give simple expressions for their asymptotic behaviour which show a lot more clearly why some primitives are better suited for certain ZK systems than others. For example, it will be very clear why Rescue is excellent in AIR (which it was designed for), but not in R1CS.

4.1 Rescue

For this section we focus on Rescue-Prime, whose definition we recalled in Section 2.1. Let n_r be the number of rounds and b be the number of branches. For any k , solving the CICO- k problem involves a system of $k + b \cdot n_r$ equations and variables. Of those equations, $b \cdot n_r$ of them have degree α (they correspond to the intermediate variables added after $x \mapsto x^{1/\alpha}$ and k of them are linear. Thus, the Bézout bound gives us the following, for any k :

$$D_I \leq \alpha^{b \cdot n_r}. \quad (2)$$

Experimental results from [8, Table 5] suggest that this bound is, in fact, tight. So, it will be the value we use, accounting for round skips. A 1-round skip is described in [8, Section 4.4] in the case where $b = 3$ and $k = 1$. A more general way to skip the first round is described in [11]. Note that this is possible due to the fact that in Rescue-Prime, the first layer is the Sbox layer, which does not make the CICO problems harder at all: this mistake was changed in the more recent descendants of Rescue, RPO [4] and XHash [3]. Therefore, we take $D_I = \alpha^{b \cdot (n_r - 1)}$.

Let $\ell(\alpha)$ is the smallest number of multiplications required to compute x^α in the R1CS model (see Appendix B for more details). The cost functions for Rescue are as follows:

$$\begin{aligned} C_{R1CS}(\alpha, b, n_r) &= 2 \cdot \ell(\alpha) \cdot b \cdot n_r, \\ C_{AIR}(\alpha, b, n_r) &= \alpha \cdot b \cdot n_r. \end{aligned}$$

Now, taking the first round skip into account, we can compute the ALFOMs:

$$\begin{aligned}\eta_{R1CS}(\alpha, b, n_r) &= \frac{\log_2(\alpha)}{2 \cdot \ell(\alpha)} \left(1 - \frac{1}{n_r}\right), \\ \eta_{AIR}(\alpha, b, n_r) &= \frac{\log_2(\alpha)}{\alpha} \left(1 - \frac{1}{n_r}\right).\end{aligned}$$

The $1/n_r$ in these expressions is due to the first round skip. We observe that:

- The ALFOMs does not depend on the number of branches, and the part of the expression dependent on n_r converges to 1.
- In the R1CS case, the $\log_2(\alpha)/\ell(\alpha)$ factor is inherent to the fact that we are not using quadratic functions, and therefore ALFOMs cannot reach the optimal value of 1 (see Appendix B). More importantly, the $1/2$ factor is essentially due to the fact that to get one equation of degree α , we need two Sboxes: one of degree α and one of inverse degree α . This is an inherent inefficiency of Rescue (in the R1CS case, as Rescue was not designed specifically for it), which approximately translates to double the constraints compared to a theoretical optimal permutation of high degree.
- For R1CS, $\log_2(\alpha)/\ell(\alpha) < 1$, but some values of α can make it approach 1.
- On the other hand, in the AIR case, there is no factor $1/2$. The term α instead of $\ell(\alpha)$ might be surprising at first, as $\ell(\alpha)$ is more comparable to $\log_2(\alpha)$, but recall that the cost functions for R1CS and AIR are of a different nature and shouldn't be compared directly. Note that smaller values of α give better ALFOMs, and the most efficient value of α for Rescue is 3. There is indeed a clear difference between $\alpha = 3$ and $\alpha = 5$ in [12, Table 2]. In fact, for $\alpha = 3$, without round skips, Rescue is AIR-optimal (see Proposition 4).

4.2 Poseidon

In this section, we will mainly consider Poseidon [24], whose definition we recalled in Section 2.1, rather than Poseidon2 [25], the latest iteration, because more detailed cryptanalysis exists for it, though we expect Poseidon2 to have a very similar analysis. While every round has total degree α , this does not necessarily mean that, when solving a CICO- k problem, we have $D_I = \alpha^{k(R_F + R_P)}$. The authors of [9] construct a system where we have

$$D_I = \alpha^{k(R_F - 2) + R_P + 1}.$$

The $(R_F - 2)$ term comes from the fact that, essentially, two rounds are skipped. We expect that in Poseidon2, where the authors added an additional linear layer to prevent one round skip, this term would be replaced by $(R_F - 1)$ (although a rigorous justification lies beyond the scope of this paper).

For the number of constraints, the R1CS complexity is equal to the number of Sboxes times $\ell(\alpha)$. A good AIR complexity requires some work: we do have

that $w = b$ and $d_{\max} = \alpha$, but T can be reduced to $R_F + \lceil R_P/b \rceil$ compared to the naive $R_F + R_P$ (see [12, Section 7.3]). Thus, we have that

$$\begin{aligned} C_{R1CS}(\alpha, b, R_F, R_P) &= \ell(\alpha) \cdot (b \cdot R_F + R_P), \\ C_{AIR}(\alpha, b, R_F, R_P) &= \alpha \cdot b \cdot \left(R_F + \left\lceil \frac{R_P}{b} \right\rceil \right). \end{aligned}$$

Putting everything together, we get the ALFOMs:

$$\begin{aligned} \eta_{R1CS}(\alpha, b, R_F, R_P, k) &= \frac{k(R_F - 2) + R_P + 1}{bR_F + R_P} \cdot \frac{\log_2(\alpha)}{\ell(\alpha)}, \\ \eta_{AIR}(\alpha, b, R_F, R_P, k) &= \frac{k(R_F - 2) + R_P + 1}{bR_F + b\lceil R_P/b \rceil} \cdot \frac{\log_2(\alpha)}{\alpha}. \end{aligned}$$

- For both R1CS and AIR, the ALFOMs increases with k , and decreases with b . This is natural: the less input variables we have, the more full rounds are “overkill”. For instance, for one variable, the output (univariate) polynomial has degree $\alpha^{R_F+R_P}$ (ignoring round skips): full rounds count as much as partial rounds for security, yet full rounds count much more. At the same time, the authors do not claim that CICO-1 is secure for the security parameter λ . We argue that the most relevant value of k in this context is the smallest k such that $p^k > 2^\lambda$ (generally, k is the capacity of the sponge).
- The most interesting “asymptotic” behaviour is when R_P tends to infinity. Indeed, the Poseidon (and Poseidon2) permutations are scaled so that the number of full rounds is small (constant in fact: $R_F = 8$ on all proposed concrete instances) compared to the number of partial rounds, which grows according to p , b , α and λ . And indeed, we have that:

$$\begin{aligned} \eta_{R1CS}(\alpha, b, R_F, R_P, k) &\underset{R_P \rightarrow \infty}{\sim} \frac{\log_2(\alpha)}{\ell(\alpha)}, \\ \eta_{AIR}(\alpha, b, R_F, R_P, k) &\underset{R_P \rightarrow \infty}{\sim} \frac{\log_2(\alpha)}{\alpha}. \end{aligned}$$

This asymptotic behaviour is, as far as we know, as good as it gets for primitives based on power- α Sboxes. And indeed, the partial rounds do achieve the “ideal” scenario where each Sbox adds an α factor to D_I . However, for practical instances, the full rounds do introduce a certain amount of inefficiency (see Table 3). Furthermore, each round has direct “low-degree”, which may introduce some additional attack vectors compared to a primitive like Rescue: a recent cryptanalytic work [26] has exploited Neptune’s (worse) property, that all rounds and their compositional inverse have low degree.

4.3 Anemoi

Anemoi is the last family of permutations that we will study. Its definition was recalled in Section 2.1. From the original paper [12], the cost functions are:

$$\begin{aligned} C_{R1CS}(\alpha, b, n_r) &= (\ell(\alpha) + 2) \cdot \frac{b}{2} \cdot n_r, \\ C_{AIR}(\alpha, b, n_r) &= \alpha \cdot b \cdot n_r. \end{aligned}$$

In the case $b = 2$ (in which case only CICO-1 instances have solutions), we use $D_I = (\alpha + 2)^{n_r}$. This value is experimentally tight and is generally an upper bound as proven in [14, Chapter 11], where the authors conjecture that for the general case, n_r can be replaced by $n_r \cdot \frac{b}{2}$. Round skips are described in [6] when $b \geq 4$, where $\frac{b}{2} - 1$ equations and variables are removed for solving the CICO-1. It is possible to generalize these skips for CICO- k for any k , where we can remove $\frac{b}{2} - \lceil \frac{k+1}{2} \rceil$ equations (see [5, App. D]), and so we take, for all parameters:

$$D_I = (\alpha + 2)^{\frac{b}{2}(n_r-1) + \lceil \frac{k+1}{2} \rceil}.$$

Notice that the previous formula also works for $b = 2$ and $k = 1$ even though there are no round skips in that case. We get the following ALFOMs formulas:

$$\begin{aligned}\eta_{R1CS}(\alpha, b, n_r, k) &= \frac{\log_2(\alpha + 2)}{\ell(\alpha) + 2} \left(1 + \frac{1}{n_r} \left(\frac{\lceil \frac{k+1}{2} \rceil}{\frac{b}{2}} - 1 \right) \right), \\ \eta_{AIR}(\alpha, b, n_r, k) &= \frac{\log_2(\alpha + 2)}{2\alpha} \left(1 + \frac{1}{n_r} \left(\frac{\lceil \frac{k+1}{2} \rceil}{\frac{b}{2}} - 1 \right) \right).\end{aligned}$$

Note that the whole part in parentheses converges to 1 as n_r tends to infinity. While round skips shouldn't be ignored, this implies that the most important parts of these formulas are the $\log_2(\alpha)/(\ell(\alpha) + 2)$ and $\log_2(\alpha)/(2\alpha)$. There are a few observations we can make:

- In the R1CS model, Anemoi's ALFOMs can converge to the optimal value of 1, as n_r tends to infinity and we choose $\alpha = 2^k + 1$ as k tends to infinity. However, practically, only very small values of α are chosen, and the ALFOMs of Anemoi is hindered by the $+2$ in the numerator which is inside of the $\log_2$ rather than outside. This is due to the fact that each round of Anemoi has $2\ell(\alpha)$ multiplications, but only multiplies the degree of the ideal by $\alpha + 2$, rather than (as a loose upper bound) 4α . The term $\alpha + 2$ is peculiar in this context: it appears because of the particular way in which leading monomials can cancel each other, as explained in [7] and [14, Chapter 11]. Indeed, we can verify experimentally that merely switching the signs in the nonlinear layer of Anemoi (so as to prevent leading monomial cancellation) can lead to a higher value for D_I .
- In the AIR model, we see that, much like other primitives, smaller values of α give a higher ALFOMs. Compared to Rescue, the primitive with the best asymptotic AIR ALFOMs (see Sec 4.1), Anemoi performs overall worse: not taking the round skips into account, we must compare $\log_2(\alpha + 2)/(2\alpha)$ and $\log_2(\alpha)/\alpha$. As α grows, the factor $1/2$ that appears in Anemoi's ALFOMs implies that Anemoi should perform only half as well as Rescue. Concretely, if we take $\alpha = 3$ (the most interesting value in this context), we have that:

$$\begin{aligned}\eta_{AIR, Rescue}(\alpha = 3, b, n_r, k) &\underset{n_r \rightarrow \infty}{\sim} 0.528, \\ \eta_{AIR, Anemoi}(\alpha = 3, b, n_r, k) &\underset{n_r \rightarrow \infty}{\sim} 0.387.\end{aligned}$$

This difference isn't really obvious in [12, Table 2], where Anemoi is listed to have less AIR constraints than Rescue in some instances with $\alpha = 3$ and a low number of branches (but it is clear that Rescue performs quite better for $\alpha = 5$, as expected). However, one should recall that the current version of Anemoi is not properly scaled with respect to D_I , which led to full-round PoSSo attacks in [7,6], particularly strong in instances with $\alpha = 3$ and a low number of branches. Assuming that Anemoi and Rescue are scaled in the same way, one should expect an efficiency ratio given by ALFOMs.

5 Optimal Constructions: the MOIRAI

In this section, we study constructions that we prove or conjecture to be optimal for multiple variants of the ALFOMs metric. Beyond their potential usability as ZK-friendly hash function component, these round functions showcase how ALFOMs and their maximisation also provide a useful guideline when designing round functions. The three families of permutations we introduce are called the MOIRAI: CLOTHO, LACHESIS, and ATROPOS⁵.

5.1 Defining the MOIRAI

Definition 7 (MOIRAI). *Given a prime number p , an even number of branches $2b$, a number of rounds n_r , a full rank matrix M , and a function $S : \mathbb{F}_p \rightarrow \mathbb{F}_p$, we define a MOIRAI as a permutation of $\mathbb{F}_p^b$ equal to*

$$\mathcal{A}_{n_r} \circ S \circ \mathcal{A}_{n_r-1} \circ S \circ \dots \circ \mathcal{A}_1 \circ S \circ \mathcal{A}_0,$$

where, for $0 \leq j \leq n_r$, $\mathcal{A}_j$ denotes the multiplication by M followed by the addition of round constant $c_{i,j}$ on the i -th branch for $1 \leq i \leq b$, and S maps $(x_1, \dots, x_b, y_1, \dots, y_b)$ of $2b$ elements to $(x_1, \dots, x_b, y_1 + S(x_1), \dots, y_b + S(x_b))$.

This overall structure is summarized in Figure 1a. We define three particular subfamilies of the MOIRAI:

1. CLOTHO, for which $S : x \mapsto x^\beta$ for any $\beta \geq 2$, not necessarily coprime to $p - 1$.
2. LACHESIS, for which $S : x \mapsto x^{1/\alpha}$ for some α coprime to $p - 1$.
3. ATROPOS, the ones for which $S : x \mapsto (x^\delta - \theta)^{-1}$, where $\delta \geq 2$ **must not be** coprime to $p - 1$ (such as all powers of 2) and θ is not a δ -th power in $\mathbb{F}_p$. Such a value exists if and only if δ is not coprime with $p - 1$.

Different Linear Layers. By using particular structures for the matrix $\mathbf{M}$, we obtain MOIRAI variants that are essentially Feistel networks (up to a renumbering of their input words). More generally, we will focus on two different structures for this matrix corresponding to the following two cases.

⁵ We followed the tradition of using names from Greek mythology. These are the three sisters that personify destiny.

SPN case. We say that a MOIRAI *has MDS (affine) layers* when the matrix $\mathbf{M}$ itself is MDS.

Feistel Network case. We say that it has Half-MDS (affine) layers if it is defined using two MDS matrices $\mathbf{M} \in \mathbb{F}_p^{2b \times 2b}$ and $\mathbf{M}' \in \mathbb{F}_p^{b \times b}$ as follows. The affine layer $\mathcal{A}_j$ for $0 < j < n_r$ is written $\mathcal{A}_j : \mathbf{v}_L \parallel \mathbf{v}_R \mapsto (\mathbf{v}_R \parallel \mathbf{M}' \times \mathbf{v}_L) + \mathbf{c}_j$, where $\mathbf{v}_L$ and $\mathbf{v}_R$ are vectors of size b , $\mathbf{c}_j = (c_{1,j}, \dots, c_{2b,j})$, while the first and last affine layers (i.e. for $j \in \{0, n_r\}$) are such that $\mathcal{A}_j = \mathbf{M} + \mathbf{c}_j$.

Table 2: Qualitative comparison of the MOIRAI (MDS or Half-MDS layers).
*: Not as fast as CLOTHO, but faster than LACHESIS.

	Computable D_I	S has high degree	S is injective	Can always have $d_{\max} \leq 4$	Fast evaluation
CLOTHO	✓	✗	✗	✓	✓
LACHESIS	✓	✓	✓	✗	✗
ATROPOS	✗	✓	✗	✓	✗*

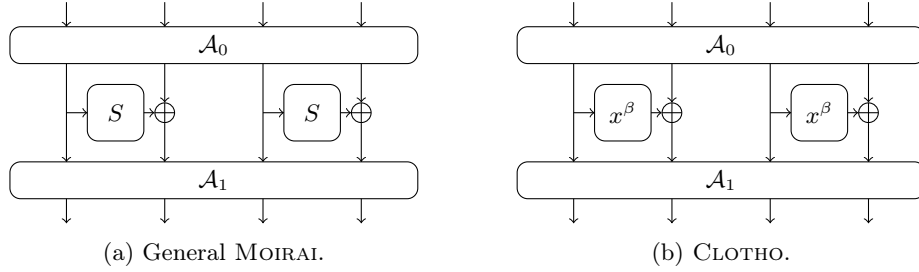


Fig. 1: First three layers of a MOIRAI.

Properties of Specific MOIRAI. Giving a full specification of the MOIRAI falls outside of the scope of this paper, and while it may be an interesting future work, the point of this section is only to study them as “ALFOMs-efficient” primitives. In the following subsections, we will show that the CLOTHO family can reach the optimal ALFOMs for both R1CS and AIR for CICO- b (but not at the same time), where the total number of branches is $2b$. We also show that LACHESIS can be AIR-optimal, but not R1CS-optimal. Lastly, we show that ATROPOS has some practical advantages over LACHESIS, although computing precisely the value of D_I is infeasible at this stage. In some cases where we conjecture the Bézout bound on D_I to be tight, we show that ATROPOS can be AIR-optimal, but not R1CS-optimal, like LACHESIS. We consider LACHESIS and

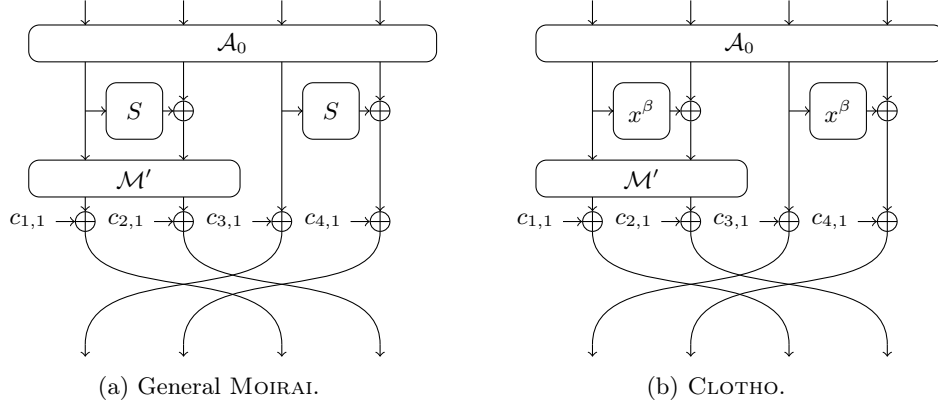


Fig. 2: First three layers of a MOIRAI with half-MDS layers.

ATROPOS as interesting alternatives whose nonlinear layers do not have a low degree. We show in particular that their ALFOMs can be arbitrarily close to the optimal value, as the security level tends to infinity.

5.2 Analysis of CLOTHO

Recall (Definition 7) that CLOTHO features a nonlinear layer that applies the $(x, y) \mapsto (x, y + x^\beta)$ in parallel, for some nonzero integer β .

Proposition 5 (D_I of CLOTHO). *Given an instance of CLOTHO over $2b$ branches with either MDS or half-MDS layers, consider the CICO- b problem for $k \leq b$. It can be solved with a polynomial system of $b+n_r b$ equations and variables whose ideal degree is*

$$D_I = \beta^{b \cdot n_r}. \quad (3)$$

Proof. We can compute D_I exactly by constructing a polynomial model that solves the CICO- b problem, and that is already a Gröbner basis in grevlex order. To this end, we follow the Freelunch [7] method. Here we do the simplest case, where the CLOTHO instance we consider has half-MDS affine layers. The MDS affine layers case also works, but it is a bit more technical, and we describe the proof in Appendix C. Denote $\mathbf{M}$ and $\mathbf{M}'$ as the matrices that define the MDS and half-MDS layers respectively. First, we represent the input of the CICO- b problem as the sum $x_{1,1}\mathbf{v}_{1,0} + \dots + x_{b,1}\mathbf{v}_{b,0}$. Each vector $\mathbf{v}_{i,0} = (v_{i,0,1}, \dots, v_{i,0,b}, 0, \dots, 0) \in \mathbb{F}_p^{2b}$ is defined by a linear system of the form

$$\mathbf{M} \cdot \mathbf{v}_{i,0} = (\underbrace{0, \dots, 0}_i, \underbrace{1, 0, \dots, 0}_{b-i}, \underbrace{*, \dots, *}_b)$$

where 1 is at the i -th coordinate⁶, i.e. $\mathbf{M} \cdot \mathbf{v}_{i,0}$ is equal to 0 on all left-side coordinates except at i where it is equal to 1. Such a $\mathbf{v}_{i,0}$ always exists because

⁶ Here, the index of the first coordinate is 1.

$\mathbf{M}$ is MDS. Now, we proceed by induction. Let $1 \leq j \leq n_r$ and $1 \leq i \leq b$. Suppose that, at the input of the j -th $\mathcal{S}$ layer, the i -th branch is a degree-1 polynomial denoted $L_{i,j}$ in variables $\{x_{i,j-1}\} \cup \{x_{i',j'} : 1 \leq i' \leq b, 0 \leq j' < j-1\}$. As we have seen, this is true for $j = 1$. We then introduce the variable $x_{i,j}$ for $1 \leq i \leq b$ at the output of the $\mathcal{S}$ function of the i -th branch. Note that in the following round, due to the structure of the half-MDS affine layers, $x_{i,j}$ will still only be on the i -th branch, which validates our induction hypothesis. It verifies the equation $x_{i,j} = L_{i,j}(\{x_{i,j-1}\} \cup \{x_{i',j'} : 1 \leq i' \leq b, 0 \leq j' < j-1\})^\beta$. The leading monomial of this equation is $x_{i,j-1}^\beta$, assuming that we use the grevlex order with a lexicographical order such that, if $j_1 < j_2$, then $x_{i_1,j_1} < x_{i_2,j_2}$ for all i_1, i_2, j_1, j_2 . By induction, we then fully define the model. The last b equations are affine and can be reduced so that their leading monomials are precisely $\{x_{i,n_r} : 1 \leq i \leq b\}$. We know that this is always possible because the last layer is an MDS layer, and the $\{x_{i,n_r} : 1 \leq i \leq b\}$ variables are diffused to all of the output branches. In doing so, we have ensured that the set of leading monomials of the model with the appropriate grevlex order is exactly $\{x_{i,j}^2 : 1 \leq i \leq b, 0 \leq j < n_r\} \cup \{x_{i,n_r} : 1 \leq i \leq b\}$. They are coprime, therefore they define a Gröbner basis (see [7, Prop. 4]). Its ideal degree is the product of the degrees of the leading monomials, and we deduce D_I . $\square$

Proposition 6 (Costs of CLOTHO). *Given an instance of CLOTHO over $2b$ branches, it is possible to encode it using R1CS or AIR with the following costs:*

$$C_{R1CS}(\beta, b, n_r) = \ell(\beta) \cdot b \cdot n_r, \quad C_{AIR}(\beta, b, n_r) = \beta \cdot 2b \cdot \left\lceil \frac{n_r}{2} \right\rceil.$$

Proof. The R1CS cost is immediate: each of the $b \cdot n_r$ Sboxes adds $\ell(\beta)$ quadratic constraints. For the AIR complexity, we adopt a similar modelling strategy as in Rescue [2], since $\mathcal{S}$ and its inverse have degree β . We introduce new variables every two rounds, such that the equations are defined by $\mathcal{S} \circ \mathcal{A}_{2i-2}(\mathbf{x}_i) = \mathcal{A}_{2i-1}^{-1} \circ \mathcal{S}^{-1}(\mathbf{x}_{i+1})$, for $0 \leq i \leq (\lceil n_r/2 \rceil - 2)$. For the last round, the equations are

$$\mathcal{A}_{2\lceil n_r/2 \rceil - 1} \mathcal{S} \circ \mathcal{A}_{2\lceil n_r/2 \rceil - 2}(\mathbf{x}_{\lceil n_r/2 \rceil - 1}) = \begin{cases} \mathcal{S}^{-1} \circ \mathcal{A}_{n_r}^{-1}(\mathbf{x}_{n_r/2}) & \text{if } n_r \text{ is even,} \\ \mathbf{x}_{(n_r+1)/2} & \text{if } n_r \text{ is odd,} \end{cases}$$

where $\mathbf{x}_0$ is the input and $\mathbf{x}_{\lceil n_r/2 \rceil}$ is the output. Altogether, we have $d_{\max} = \beta$, $w = 2b$ and $t = \lceil n_r/2 \rceil$, which we multiply to get C_{AIR} . $\square$

Thus, using the models of Propositions 5 and 6, we can compute CLOTHO's ALFOMs.

Corollary 1. *For an instance of CLOTHO with $2b$ branches, $k = b$, and whose affine layers are MDS or half-MDS, the ALFOMs are*

$$\eta_{R1CS}(\beta, b, n_r) = \frac{\log_2(\beta)}{\ell(\beta)}, \quad \eta_{AIR}(\beta, b, n_r) = \begin{cases} \frac{\log_2(\beta)}{\beta} & \text{if } n_r \text{ even,} \\ \frac{\log_2(\beta)}{\beta} \left(1 - \frac{1}{n}\right) & \text{otherwise.} \end{cases}$$

Proof of Corollary 1. The formulas can be deduced from Propositions 5 and 6. For $m \geq 1$ and $\beta = 2^m$, we have that $\log_2(\beta) = \ell(\beta) = m$, which implies that $\eta_{R1CS}(2^m, b, n_r) = 1$. For AIR, we have that

$$\eta_{AIR}(\beta, b, n_r) = \frac{\log_2(\beta) \cdot b \cdot n_r}{\beta \cdot 2b \cdot \lceil n_r/2 \rceil},$$

and $n_r/(2\lceil n_r/2 \rceil)$ is equal to 1 if n_r is even, and $1 - 1/n_r$ if n_r is odd. $\square$

Regarding optimality, we know from Propositions 3 and 4 that η_{R1CS} and η_{AIR} are respectively upper-bounded by 1 and $\log_2(3)/3$. For R1CS, the values of β that maximize η_{R1CS} are the powers of 2, and if additionally n_r is even, then CLOTHO is R1CS-optimal, as for any $m \geq 1$, $\eta_{R1CS}(2^m, b, n_r) = 1$. For AIR, the value of β that maximizes η_{AIR} is 3, for which CLOTHO is AIR-optimal. The second-best are 2 and 4, which also yield R1CS-optimal instances:

$$\eta_{AIR}(3, b, n_r) = \frac{\log_2(3)}{3} \approx 0.53, \quad \eta_{AIR}(2, b, n_r) = \eta_{AIR}(4, b, n_r) = 0.5.$$

When $\gcd(\beta, p-1)$ is big. The $x \mapsto x^\beta$ function is not necessarily a permutation. Depending on the factorisation of $p-1$, it may be, at worst, a β -to-1 function. If p is small and β is big, this may lead to some weaknesses against non-PoSSo-based cryptanalysis. While outside the scope of this paper, it is still worth noting.

5.3 Analysis of LACHESIS

CLOTHO, while R1CS-optimal in some cases, has the potentially unfortunate property of being of overall low degree (compared to random permutations of $\mathbb{F}_p^n$). Indeed, the quadratic permutations described currently in the literature [27, 22] are all Feistel or Lai-Massey constructions, which have low-degree inverse. This can enable up some attack vectors, as evidenced by a recent paper [26] which exploits a similar property in Neptune.

This observation motivated our design of LACHESIS, which uses $x \mapsto x^{1/\alpha}$ as its non-linear layer. The inverse being of low degree, it is easy to derive low-degree equations for both AIR and R1CS. However, since the degree of the evaluation is very high, the permutation itself will quickly reach a very high degree.

The use of the inverse of low degree monomial is not new, it was already introduced by the designers of Rescue. However, this approach has a cost both in terms of R1CS constraints, and in terms of field dependence. Indeed, a high degree function enabling a low degree verification ultimately relies on the inverse of low degree monomials (in Rescue, Anemoi and Griffin), or on the inverse function (Vision). However, neither can allow a good ALFOM for AIR since the addition chains for the exponent are not of optimal length. In light of this observation, we make the following conjecture:

Conjecture 1. Any permutation that is R1CS-optimal, that is, for which $\eta_{R1CS} = 1$ for some set of parameters, must be a composition of permutations that are of low degree, and whose inverse have a low degree. In other words, it is impossible to have both R1CS-optimality and a high-degree round function.

Another key difference between the use of non-linear functions of low degree and non-linear functions with low degree verification lies in the cryptanalyses they allow. In the case of a low-degree evaluation, it is possible to directly solve a univariate system obtained by simply applying the permutation to a symbolic variable. This is known to have a time complexity quasi-linear in D_I . On the other hand, forcing the introduction of intermediate variables may also a priori force cryptanalysts to use algorithms with a complexity proportional to D_I^ω or higher. Still, recent breakthroughs in Gröbner Basis and resultant-based attacks [7,33,6] achieve a complexity which is lower than $\tilde{O}(D_I^\omega)$ due to dedicated computer algebra techniques. Thus, in the remainder of this paper, we assume that all primitives are simply scaled so that $D_I \geq 2^\lambda$.

Proposition 7 (D_I of LACHESIS). *Given an instance of LACHESIS over $2b$ branches, consider the CICO- b problem for $k \leq b$. It can be solved with a system of $b + n_r b$ equations and variables whose ideal degree is: $D_I = \alpha^{b \cdot n_r}$.*

Proof. To solve CICO- b on LACHESIS, we define variables $x_{1,0}, \dots, x_{b_0}$ at the input. Then, we add an intermediate variable at the output of each $x \mapsto x^{1/\alpha}$ function. We obtain, for each $1 \leq i \leq b$ and $1 \leq j \leq n_r$, an equation of the form $x_{i,j}^\alpha = L_{i,j}(x_{i',j'} : 1 \leq i' \leq b, 1 \leq j' < j)$, whose leading monomial is $x_{i,j}^\alpha$ in grevlex order. Finally, we get b affine equations at the output, which we can apply gaussian reduction to, and in a lexicographical order where $x_{i,0} > x_{i',j'}$ for all $1 \leq i, i' \leq b$ and $1 \leq j' \leq n_r$, we get $x_{i,0}$ as leading monomials of the system for all i . Thus we have a system of $b + b \cdot n_r$ equations and variables, where the leading monomials are pairwise coprime, and so the degree of the ideal is equal to the product of their degrees. $\square$

Proposition 8 (Costs of LACHESIS). *Given an instance of LACHESIS over $2b$ branches, it is possible to encode it to an R1CS and an AIR with the following costs:*

$$C_{R1CS}(\alpha, b, n_r) = \ell(\alpha) \cdot b \cdot n_r, \quad C_{AIR}(\alpha, b, n_r) = \alpha \cdot 2b \cdot \left\lceil \frac{n_r}{2} \right\rceil.$$

Proof. For R1CS, the number of $x \mapsto x^{1/\alpha}$ functions is $b \cdot n_r$, and each one requires $\ell(\alpha)$ constraints. For AIR, we define a model with the following method: let us define the same variables as for the AIR model of CLOTHO (see the proof of 6). Let $0 \leq j \leq (\lceil n_r/2 \rceil - 2)$. We can define $2b$ independent equations of degree α between the variables of $\mathbf{x}_j$ and those of $\mathbf{x}_{j+1}$ with the following approach: consider part of the permutation $\mathcal{S} \circ \mathcal{A}_{j+1} \circ \mathcal{S}$, and let y_i and z_i for $1 \leq i \leq 2b$ be variables at the input and output it: they are degree-1 expressions in the variables of $\mathbf{x}_j$ and those of $\mathbf{x}_{j+1}$ respectively. Let t_i be the output of the $2i$ -th branch after the first $\mathcal{S}$ layer, so that $t_i = \mathcal{S}(y_{2i-1}) + y_{2i}$. We have that, for every $1 \leq i \leq b$, $z_{2i-1} = L_i(y_{2i'-1}, t_{i'} : 1 \leq i' \leq b)$, where L_i is an affine function. Now, we can rearrange these b affine equations in $3b$ variables, in order to express the $t_{i'}$ variables as affine functions of the others: for every $1 \leq i \leq b$, there exists an affine polynomials R_i such that $t_i = R_i(y_{2i'-1}, z_{2i'-1} : 1 \leq i' \leq b)$. By substituting t_i with this affine expression, we can derive an equation of degree

α : $y_{2i-1} = (R_i(y_{2i'-1}, z_{2i'-1} : 1 \leq i' \leq b) - y_{2i})^\alpha$. Symmetrically, by applying the same method on the $\mathcal{S}$ situated on the “other side” of the $\mathcal{A}_{j+1}$ layer, we also get b equations of the shape $z_{2i-1} = (R'_i(z_{2i'-1}, y_{2i'-1} : 1 \leq i' \leq b) - z_{2i})^\alpha$. Finally, by replacing the y and z variables by their affine expressions in the $\mathbf{x}_j$ and $\mathbf{x}_{j+1}$ variables respectively, we get the final $2b$ equations of degree α that are part of our AIR. When n_r is odd, for the final round, a slightly different (but simpler) approach lets us represent the final round with $2b$ equations of degree α between the variables of $\mathbf{x}_{n_r-1}$ and $\mathbf{x}_{n_r}$, much like with CLOTHO. Finally, we get an AIR with $d_{\max} = \alpha$, $w = 2b$ and $t = \lceil n_r/2 \rceil$. $\square$

Corollary 2 (ALFOMs of LACHESIS). *Given the models of Propositions 7 and 8, we have that, for an instance of LACHESIS with $2b$ branches and $k = b$, its ALFOMs are*

$$\eta_{R1CS}(\alpha, b, n_r) = \frac{\log_2(\alpha)}{\ell(\alpha)},$$

$$\eta_{AIR}(\alpha, b, n_r) = \begin{cases} \frac{\log_2(\alpha)}{\alpha} & \text{if } n_r \text{ is even,} \\ \frac{\log_2(\alpha)}{\alpha} \left(1 - \frac{1}{n_r}\right) & \text{if } n_r \text{ is odd.} \end{cases}$$

For R1CS, fix $m \geq 2$. The value of α that maximizes η_{R1CS} , under the condition that each round has at most $b \cdot m$ quadratic constraints, is $2^{m-1} + 1$ (see Proposition 11 from Appendix B) when it is coprime with $p - 1$. For this value, and for n_r even,

$$\eta_{R1CS}(2^{m-1} + 1, b, n_r) = \frac{\log_2(2^{m-1} + 1)}{m} \underset{m \rightarrow \infty}{\sim} 1.$$

For AIR, the value of α that maximizes η_{AIR} is 3, when it is coprime with $p - 1$. For this value, and n_r even, $\eta_{AIR}(3, b, n_r) = \log_2(3)/3 \approx 0.53$, in which case LACHESIS is AIR-optimal.

5.4 Analysis of ATROPOS

ATROPOS is the last MOIRAI we define. It uses the high-degree $x \mapsto 1/(x^\delta - \theta)$ functions, where θ is not a δ -th power in $\mathbb{F}_p$ (thus, δ must not be coprime to $p - 1$). The inverse function makes ATROPOS a bit harder to analyze, and it seems like this construction is redundant compared to LACHESIS, in that it also has high-degree Sboxes at the cost of non-optimality with respect to R1CS. However, we identify some notable advantages compared to LACHESIS and primitives that use $x \mapsto x^{1/\alpha}$ in general:

- For R1CS, we can always choose δ as a multiple of 2, so we can always have a cost of m multiplications to multiply D_I by $2^{m-1} + 1$. This mimics the best choice of $\alpha = 2^{m-1} + 1$ in LACHESIS. However, in LACHESIS (and similarly, primitives like Rescue [2]), we may not be able to choose the most interesting low values for α : $\alpha = 3$, $\alpha = 5$, or $\alpha = 9$ —the highest exponents to require respectively 2, 3 and 4 multiplications. In fact, none of these can be chosen for $p = 2^{64} - 2^{32} + 1$, which is why XHash [3] has to resort to $\alpha = 7$.

- For AIR, similarly, the most interesting value, $d_{\max} = 3$, can always be chosen. That is not the case for LACHESIS, Rescue, XHash, Anemoi or Griffin for the following widely used primes: $2^{64} - 2^{32} + 1$ [3], the Mersenne prime $2^{31} - 1$, and the primes of the curves BLS12-381 and BN-254 [12].
- In $\mathbb{F}_p$, with modular arithmetic, the inverse function can be computed more efficiently than high-degree power maps, using the extended Euclidean algorithm⁷. Additionally, [13, Sec. 2.5.1] describes an efficient algorithm to compute multiple modular inversions modulo the same number at the same time.

On the inverse function. The “inverse” function is a bit tricky to work with. Indeed, we call it like that (and the authors of Vision even write it x^{-1}) even though 0 has no inverse. Implicitly, we just map 0 to 0 to complete the function, which actually yields the function $x \mapsto x^{p-2}$. This potentially induces a gap between the constraint system and cryptanalysis: given an equation $y = x^{p-2}$, the constraint system must model it as $yx^2 = x$, instead of $yx = 1$, to take into account the case where $x = 0$. On the other hand, a cryptanalyst may assume that the inputs of the inverse Sboxes are never equal to 0 (which becomes more likely as p grows) and solve for equations of degree 2 instead of 3. This issue can be resolved by making sure that inverse Sboxes never receive 0 as an input. For instance, we can first map the input x to $x^2 - \theta$ (where θ is not a square in the base field), and apply the inverse to the result. This idea led us to define ATROPOS.

Proposition 9 (D_I of ATROPOS). *Given an instance of ATROPOS over $2b$ branches, the CICO- b problem for $k \leq b$ can be solved with a polynomial system of $b + n_r b$ equations and variables whose ideal degree verifies: $D_I \leq (\delta + 1)^{b \cdot n_r}$.*

Proof. Since we are not explicitly building a Gröbner basis, the proof is quite simple. We define the input state for the CICO- b as $(x_{1,0}, \dots, x_{b,0}, 0, \dots, 0)$. Let $1 \leq i \leq b$ and $1 \leq j \leq n_r$. We define the variable $x_{i,j}$ as the output of the i -th $x \mapsto 1/(x^\delta - \theta)$ function of the j -th $\mathcal{S}$ layer. Inductively, the input of this function will be a degree-1 polynomial in all variables defined over the previous round, which we call $L_{i,j}$. Then, the corresponding equation has the shape

$$x_{i,j} \left(L_{i,j}(x_{i',j'} : 1 \leq i' \leq b, 0 \leq j' < j)^\delta - \theta \right) = 0.$$

Each of these $b \cdot n_r$ equations has total degree $\delta + 1$. Finally, b degree-1 equations that model the output of the CICO- b problem complete the system. Therefore, we deduce the result from the Bézout bound: D_I is at most the product of the total degrees of all of the equations. $\square$

⁷ Concretely, we can measure the difference in sagemath: for the p of BLS12-381, on a laptop, computing a million `inverse_mod(_,p)` takes about 3.0s, whereas a million `pow(_,alphainv,p)`, with `alphainv` the precomputed modular inverse of 5, takes about 24.0s. For $p = 2^{31} - 1$, the difference is smaller: 1.0s for the former, 2.0s for the latter.

Experimental verifications. The results of our experiments are summarized in Appendix E (Table 4 in particular). We notice a big difference between MDS layers and non-MDS layers: the Bézout bound is tight in all of the instances we verified with MDS layers, but those that have non-MDS layers have lower D_I than expected starting at $n_r = 3$. Nevertheless, we conjecture that in the MDS layers case, the Bézout bound is tight in general for the model we described. Verifying that conjecture and/or explicitly describing a Gröbner basis for this system for various kinds of affine layers remain open questions, and we leave them for future work. For the remainder of this section (in particular, to compute ATROPOS’s ALFOMs), we will assume that the Bézout bound is, in fact, tight, when using MDS layers.

Proposition 10 (Costs of ATROPOS). *An instance of ATROPOS over $2b$ branches, can be encoded it into an R1CS or an AIR with the following costs:*

$$C_{R1CS}(\delta, b, n_r) = (\ell(\delta) + 1) \cdot b \cdot n_r, \quad C_{AIR}(\delta, b, n_r) = (\delta + 1) \cdot 2b \cdot \left\lceil \frac{n_r}{2} \right\rceil.$$

Proof. For R1CS, the input x_1 and the output x_2 of the $x \mapsto (x^\delta - \theta)^{-1}$ function are linked by the equation $x_2 \cdot (x_1^\delta - 1) = 1$. Furthermore, x_1^δ can be computed from x_1 using $\ell(\delta)$ quadratic constraints, and one additional quadratic constraint is used to multiply $x_1^\delta - \theta$ by x_2 . For AIR, we can reuse the same method that we described for LACHESIS (see the proof of Proposition 8). Using the same notations, the only difference is that, instead of the equation $y_{2i-1} = (R_i(y_{2i'-1}, z_{2i'-1}) - y_{2i})^\alpha$ of degree α (and its symmetrical equivalent), we have instead the equation $(y_{2i-1}^\delta - \theta)(R_i(y_{2i'-1}, z_{2i'-1}) - y_{2i}) = 1$, which is of degree $\delta + 1$. Thus, we get that $d_{\max} = \delta + 1$, $w = 2b$ and $t = \lceil n_r/2 \rceil$. $\square$

Corollary 3 (ALFOMs of ATROPOS). *Consider an instance of ATROPOS with $2b$ branches and $k = b$. Suppose that the Bézout bound for D_I , given in Prop. 9, is an equality. Then, ATROPOS’s ALFOMs are*

$$\eta_{R1CS}(\delta, b, n_r) = \frac{\log_2(\delta + 1)}{\ell(\delta) + 1},$$

$$\eta_{AIR}(\delta, b, n_r) = \begin{cases} \frac{\log_2(\delta + 1)}{\delta + 1} & \text{if } n_r \text{ is even,} \\ \frac{\log_2(\delta + 1)}{\delta + 1} \left(1 - \frac{1}{n_r}\right) & \text{if } n_r \text{ is odd.} \end{cases}$$

For R1CS, like for LACHESIS, given a fixed number $b \cdot m$ of quadratic constraints per round, the value of δ that maximizes η_{R1CS} is 2^{m-1} . For this value, and for n_r even,

$$\eta_{R1CS}(2^{m-1}, b, n_r) = \frac{\log_2(2^{m-1} + 1)}{m} \underset{m \rightarrow \infty}{\sim} 1.$$

For AIR, the value of δ that maximizes η_{AIR} is 2. For this value, when n_r is even, we have $\eta_{AIR}(2, b, n_r) = \frac{\log_2(3)}{3} \approx 0.53$, meaning that ATROPOS is AIR-optimal.

6 Conclusion

The rigid framework provided by the ALFOMs is a powerful tool. We can now compare most primitives discussed in this paper in a fair way that takes into account both their security and their performance. Given a security parameter λ , a number of branches $2b$, considering CICO- b (i.e. $k = b$), and all primitive-wise parameters aside from n_r , define n_r as $n_r = \min\{n_r : D_I(n_r, k = b) \geq 2^\lambda\}$. Naturally, this definition of n_r is much too simple for a full specification: marginal rounds should always be added, and in some cases designers may want to argue in favor of a more aggressive criterion on D_I , such as $D_I^2 \geq 2^\lambda$. Nevertheless, we consider that the resulting comparison remains relevant.

The results are summarized in Table 3. Note that we always use the best known round skips to compute D_I , but there may exist better round skips that are not known yet. That is why we focus on CICO- b : as the attacker has much less freedom for this problem, we are more confident that our analysis does not miss any significant round skip.

Table 3: Costs with n_r computed so that $D_I \geq 2^{128}$. Next to the cost, the relevant ALFOMs is shown in parenthesis. The best costs per column are in bold. All Bézout bounds are assumed to be tight.

		R1CS cost				AIR cost			
		$b = 2$	$b = 4$	$b = 6$	$b = 8$	$b = 2$	$b = 4$	$b = 6$	$b = 8$
CLOTHO	$\beta = 2$	128 (1.0)	128 (1.0)	132 (1.0)	128 (1.0)	256 (0.50)	256 (0.50)	264 (0.50)	256 (0.50)
	$\beta = 3$	164 (0.79)	168 (0.79)	168 (0.79)	176 (0.79)	252 (0.52)	264 (0.50)	252 (0.53)	288 (0.48)
LACHESIS	$\alpha = 3$	164 (0.79)	168 (0.79)	168 (0.79)	176 (0.79)	252 (0.52)	264 (0.50)	252 (0.53)	288 (0.48)
	$\alpha = 5$	168 (0.77)	168 (0.77)	180 (0.77)	168 (0.77)	280 (0.46)	280 (0.46)	300 (0.46)	320 (0.41)
ATROPOS	$\delta = 2$	164 (0.79)	168 (0.79)	168 (0.79)	176 (0.79)	252 (0.52)	264 (0.50)	252 (0.53)	288 (0.48)
Rescue-Prime	$\alpha = 3$	336 (0.39)	336 (0.40)	336 (0.40)	352 (0.40)	252 (0.52)	252 (0.53)	252 (0.53)	264 (0.53)
	$\alpha = 5$	348 (0.37)	336 (0.39)	360 (0.39)	336 (0.39)	290 (0.45)	280 (0.46)	300 (0.46)	280 (0.46)
Anemoi	$\alpha = 3$	224 (0.58)	224 (0.58)	228 (0.57)	240 (0.57)	336 (0.39)	336 (0.39)	342 (0.38)	360 (0.38)
	$\alpha = 5$	230 (0.56)	230 (0.56)	240 (0.55)	240 (0.55)	460 (0.28)	460 (0.28)	480 (0.27)	480 (0.27)
Poseidon	$\alpha = 3$	180 (0.71)	200 (0.64)	220 (0.58)	240 (0.53)	270 (0.48)	300 (0.43)	342 (0.38)	360 (0.36)
	$\alpha = 5$	195 (0.67)	225 (0.58)	255 (0.51)	285 (0.46)	330 (0.39)	380 (0.34)	450 (0.29)	480 (0.27)

As expected, CLOTHO with $\beta = 2$ has, by far, the lowest R1CS costs (all very close to $\lambda = 128$). It is, after all, the only R1CS-optimal primitive, whereas all other primitives in the table have an η_{R1CS} of less than 0.8. For AIR, the best scores are more distributed: most primitives reach (or almost reach) the optimal cost of $\log_2(3)/3 \approx 0.53$ for these low values of α, β and δ . Note how two primitives with the same ALFOMs may not have the same costs: there is a small discrepancy due to the fact that D_I is not necessarily perfectly equal to 2^λ , which adds a small overhead.

To the best of our knowledge, the case of R1CS and AIR-friendly hash functions is the first where it is possible to identify such a clear description of the tradeoff between performance and security. Thanks to the ALFOMs, it is now possible to make absolute statements about the optimality (or lack thereof) of

a given round function, and to make fair comparison of the round functions of primitives that encompass both their performance and the security they offer.

Acknowledgements

The work of Aurélien Boeuf and Léo Perrin was supported by the European Research Council (ERC, grant agreement no. 101041545 “ReSCALE”). The work of Aurélien Boeuf was also supported in part by the French Agence Nationale de la Recherche through the SWAP project under Contract ANR-21-CE39-0012. We thank Augustin Bariant  for some editorial remarks.

References

1. Albrecht, M.R., Grassi, L., Rechberger, C., Roy, A., Tiessen, T.: MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In: Cheon, J.H., Takagi, T. (eds.) ASIACRYPT 2016, Part I. LNCS, vol. 10031, pp. 191–219. Springer, Berlin, Heidelberg (Dec 2016). https://doi.org/10.1007/978-3-662-53887-6_7
2. Aly, A., Ashur, T., Ben-Sasson, E., Dhooghe, S., Szeponiec, A.: Design of symmetric-key primitives for advanced cryptographic protocols. IACR Trans. Symm. Cryptol. **2020**(3), 1–45 (2020). <https://doi.org/10.13154/tosc.v2020.i3.1-45>
3. Ashur, T., Bhati, A.S., Kindi, A., Mahzoun, M., Perrin, L.: XHash: Efficient STARK-friendly hash function. Cryptology ePrint Archive, Report 2023/1045 (2023), <https://eprint.iacr.org/2023/1045>
4. Ashur, T., Kindi, A., Meier, W., Szeponiec, A., Threadbare, B.: Rescue-prime optimized. Cryptology ePrint Archive, Report 2022/1577 (2022), <https://eprint.iacr.org/2022/1577>
5. Bak, A., Perrin, L.: On the security of split-and-lookup-based zk-friendly primitives. IACR Transactions on Symmetric Cryptology **2025**(2), 87–123 (Jun 2025). <https://doi.org/10.46586/tosc.v2025.i2.87-123>, <https://tosc.iacr.org/index.php/ToSC/article/view/12245>
6. Bariant, A., Boeuf, A., Briaud, P., Hostettler, M., Øygarden, M., Raddum, H.: Improved resultant attack against arithmetization-oriented primitives. In: Kalai, Y.T., Kamara, S.F. (eds.) CRYPTO 2025, Part V. LNCS, vol. 16004, pp. 335–367. Springer, Cham (Aug 2025). https://doi.org/10.1007/978-3-032-01901-1_11
7. Bariant, A., Boeuf, A., Lemoine, A., Ayala, I.M., Øygarden, M., Perrin, L., Raddum, H.: The algebraic FreeLunch: Efficient Gröbner basis attacks against arithmetization-oriented primitives. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part IV. LNCS, vol. 14923, pp. 139–173. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68385-5_5
8. Bariant, A., Bouvier, C., Leurent, G., Perrin, L.: Algebraic attacks against some arithmetization-oriented primitives. IACR Trans. Symm. Cryptol. **2022**(3), 73–101 (2022). <https://doi.org/10.46586/tosc.v2022.i3.73-101>
9. Ben-Sasson, E., Goldberg, L., Levit, D.: STARK friendly hash – survey and recommendation. Cryptology ePrint Archive, Report 2020/948 (2020), <https://eprint.iacr.org/2020/948>

10. Berlekamp, E.R.: Factoring polynomials over large finite fields. In: Petrick, S.R., Sammet, J.E., Tobey, R.G., Moses, J. (eds.) Proceedings of the second ACM symposium on Symbolic and algebraic manipulation, SYMSAC 1971, Los Angeles, California, USA, March 23-25, 1971. p. 223. ACM (1971). <https://doi.org/10.1145/800204.806290>, <https://doi.org/10.1145/800204.806290>
11. Boeuf, A., Canteaut, A., Perrin, L.: Propagation of subspaces in primitives with monomial sboxes: Applications to rescue and variants of the AES. *IACR Trans. Symm. Cryptol.* **2023**(4), 270–298 (2023). <https://doi.org/10.46586/tosc.v2023.i4.270-298>
12. Bouvier, C., Briaud, P., Chaidos, P., Perrin, L., Salen, R., Velichkov, V., Willems, D.: New design techniques for efficient arithmetization-oriented hash functions: Anemoi permutations and Jive compression mode. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part III. LNCS, vol. 14083, pp. 507–539. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38548-3_17
13. Brent, R.P., Zimmermann, P.: Modern computer arithmetic (version 0.5.1). *CoRR abs/1004.4710* (2010), <http://arxiv.org/abs/1004.4710>
14. Briaud, P.: Cryptanalyse Algébrique de Schémas Post-Quantiques et Hypothèses Associées. (Algebraic Cryptanalysis of Post-Quantum Schemes and Related Assumptions). Ph.D. thesis, Sorbone University, France (2023), <https://tel.archives-ouvertes.fr/tel-04358018>
15. Campa, L., Roy, A.: Groebner basis cryptanalysis of anemoi. *Cryptology ePrint Archive*, Paper 2025/814 (2025). https://doi.org/10.1007/978-3-031-91107-1_11, <https://eprint.iacr.org/2025/814>
16. Cantor, D.G., Zassenhaus, H.: A new algorithm for factoring polynomials over finite fields. *Math. Comput.* **36**, 587–592 (1981). <https://doi.org/10.2307/2007663>
17. Faugère, J.C.: A new efficient algorithm for computing Gröbner bases (F_4). *J. Pure Appl. Algebra* **139**(1-3), 61–88 (1999). [https://doi.org/10.1016/S0022-4049\(99\)00005-5](https://doi.org/10.1016/S0022-4049(99)00005-5)
18. Faugère, J.C.: A new efficient algorithm for computing Gröbner bases without reduction to zero (F_5). In: Proceedings of the 2002 international symposium on symbolic and algebraic computation, ISSAC 2002, Lille, France, July 07–10, 2002., pp. 75–83. New York, NY: ACM Press (2002)
19. Faugère, J., Gianni, P.M., Lazard, D., Mora, T.: Efficient computation of zero-dimensional gröbner bases by change of ordering. *J. Symb. Comput.* **16**(4), 329–344 (1993). <https://doi.org/10.1006/JSC0.1993.1051>, <https://doi.org/10.1006/jsc0.1993.1051>
20. Faugère, J., Mou, C.: Sparse FGLM algorithms. *J. Symb. Comput.* **80**, 538–569 (2017). <https://doi.org/10.1016/J.JSC.2016.07.025>, <https://doi.org/10.1016/j.jsc.2016.07.025>
21. Flajolet, P., Odlyzko, A.M.: Random mapping statistics. In: Quisquater, J.J., Vandewalle, J. (eds.) EUROCRYPT’89. LNCS, vol. 434, pp. 329–354. Springer, Berlin, Heidelberg (Apr 1990). https://doi.org/10.1007/3-540-46885-4_34
22. Giordani, G., Grassi, L., Onofri, S., Pedicini, M.: Invertible quadratic non-linear functions over $\mathbb{F}_p^n$ via multiple local maps. In: El Mrabet, N., De Feo, L., Duquesne, S. (eds.) AFRICACRYPT 23. LNCS, vol. 14064, pp. 151–176. Springer, Cham (Jul 2023). https://doi.org/10.1007/978-3-031-37679-5_7
23. Grassi, L., Hao, Y., Rechberger, C., Schafneggger, M., Walch, R., Wang, Q.: Horst meets fluid-SPN: Griffin for zero-knowledge applications. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part III. LNCS, vol. 14083, pp. 573–606. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38548-3_19

24. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schafneggger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: Bailey, M., Greenstadt, R. (eds.) *USENIX Security 2021*. pp. 519–535. USENIX Association (Aug 2021), <https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>
25. Grassi, L., Khovratovich, D., Schafneggger, M.: Poseidon2: A faster version of the poseidon hash function. In: El Mrabet, N., De Feo, L., Duquesne, S. (eds.) *AFRICACRYPT 23*. LNCS, vol. 14064, pp. 177–203. Springer, Cham (Jul 2023). https://doi.org/10.1007/978-3-031-37679-5_8
26. Grassi, L., Koschatko, K., Rechberger, C.: Poseidon and neptune: Gröbner basis cryptanalysis exploiting subspace trails. *Cryptology ePrint Archive*, Paper 2025/954 (2025), <https://eprint.iacr.org/2025/954>
27. Grassi, L., Onofri, S., Pedicini, M., Sozzi, L.: Invertible quadratic non-linear layers for MPC-/FHE-/ZK-friendly schemes over $\mathbb{F}_p^n$: Application to Poseidon. *IACR Trans. Symm. Cryptol.* **2022**(3), 20–72 (2022). <https://doi.org/10.46586/tosc.v2022.i3.20-72>
28. Knuth, D.E.: *The art of computer programming, Volume II: Seminumerical Algorithms*, 3rd Edition. Addison-Wesley (1998), <https://www.worldcat.org/oclc/312898417>
29. Manterola Ayala, I.: *Stap zoo*. <https://stap-zoo.com/> (2023)
30. Moenck, R.T.: Fast computation of gcds. In: Aho, A.V., Borodin, A., Constable, R.L., Floyd, R.W., Harrison, M.A., Karp, R.M., Strong, H.R. (eds.) *Proceedings of the 5th Annual ACM Symposium on Theory of Computing*, April 30 - May 2, 1973, Austin, Texas, USA. pp. 142–151. ACM (1973). <https://doi.org/10.1145/800125.804045>, <https://doi.org/10.1145/800125.804045>
31. Parno, B., Howell, J., Gentry, C., Raykova, M.: Pinocchio: Nearly practical verifiable computation. In: *2013 IEEE Symposium on Security and Privacy*. pp. 238–252. IEEE Computer Society Press (May 2013). <https://doi.org/10.1109/SP.2013.47>
32. Szepieniec, A., Ashur, T., Dhooghe, S.: Rescue-prime: a standard specification (SoK). *Cryptology ePrint Archive*, Report 2020/1143 (2020), <https://eprint.iacr.org/2020/1143>
33. Yang, H.S., Zheng, Q.X., Yang, J., feng Liu, Q., Tang, D.: A new security evaluation method based on resultant for arithmetic-oriented algorithms. In: Chung, K.M., Sasaki, Y. (eds.) *ASIACRYPT 2024, Part VII*. LNCS, vol. 15490, pp. 457–489. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0941-3_15

A Round function figures

B On Addition Chains

When designing ZK-friendly primitives, designers often use permutations like $x \mapsto x^\alpha$ over finite fields, as they are easy to analyze and can be computed using few multiplications if α is not too big. Primitives like Rescue, Griffin, Arion and Anemoui also rely heavily on the functional inverse of $x \mapsto x^\alpha$. The designers mainly focus on the cases $\alpha = 3, 5$ or 7 , as $x \mapsto x^2$ is never a permutation over any finite field. Those permutations require, respectively, 2, 3 and 4 multiplications.

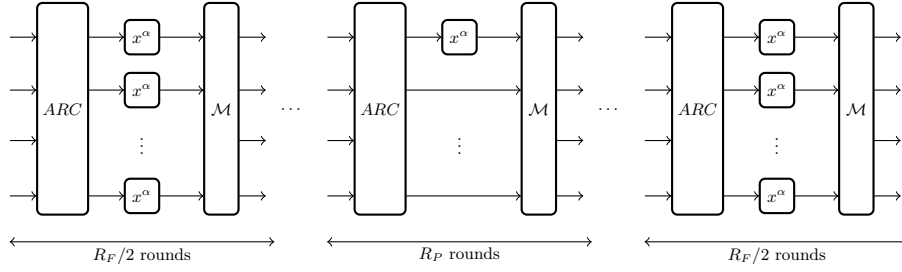


Fig. 3: Poseidon round function.

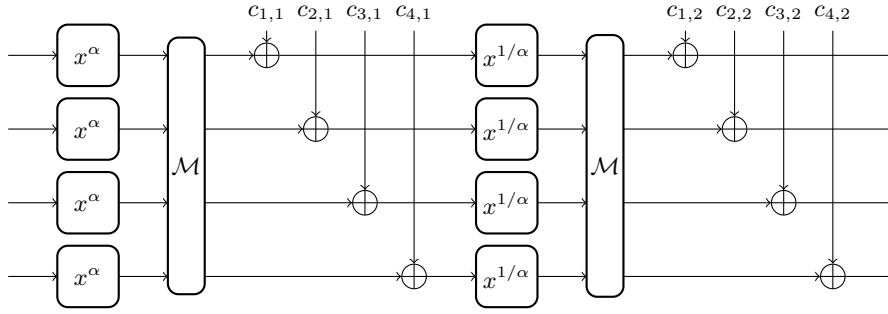


Fig. 4: Rescue-Prime round function.

The minimal number of multiplications required to compute $x \mapsto x^\alpha$ is given by the length of the smallest addition chain of α (minus one). Here, we define addition chains and their properties:

Definition 8 (Addition Chain). Let $n \geq 1$. An addition chain for n is an increasing sequence $k_0, \dots, k_m$ such that

1. $k_0 = 1$.
2. For all $0 \leq i \leq m$, k_i is the sum of two previous numbers: there exist $0 \leq j_1, j_2 < i$ such that $k_i = k_{j_1} + k_{j_2}$.
3. $k_m = n$.

We denote the shortest length of all addition chains for n (that is, the smallest m such that $k_0, \dots, k_m$ can be an addition chain) as $\ell(n)$. One important application of addition chains is that the smallest number of multiplications required to compute x^n from x is $\ell(n) - 1$.

Example 3 (Addition Chains). An addition chain for 5 could be $(1, 2, 3, 4, 5)$. Indeed, $2 = 1 + 1$, $3 = 2 + 1$, $4 = 3 + 1$ and $5 = 4 + 1$.

However, $(1, 2, 4, 5)$ and $(1, 2, 3, 5)$ are shorter (and, in fact, the shortest) addition chains for 5. This implies that x^5 can be computed from x using only 3 multiplications:

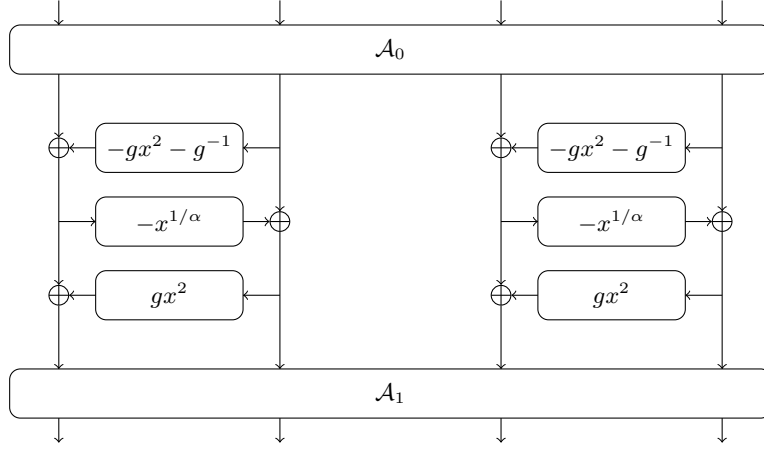


Fig. 5: Anemoi's first layers.

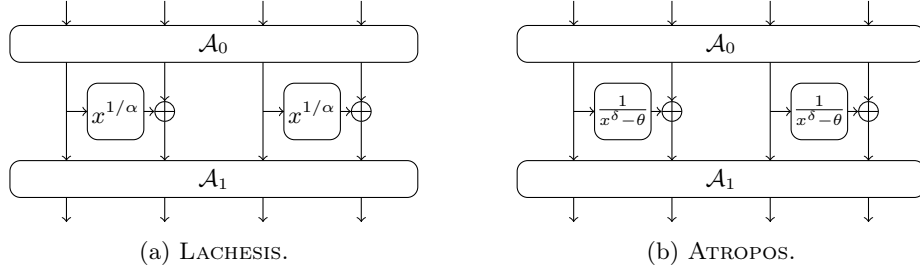


Fig. 6: First three layers of a MOIRAI.

$$\begin{aligned}
 x^2 &= x \cdot x, \\
 x^4 &= x^2 \cdot x^2, \\
 x^5 &= x^4 \cdot x.
 \end{aligned}$$

Another example: one of the shortest addition chains for 13 is (1, 2, 4, 8, 12, 13).

For a detailed resource on the subject, see [28, Section 4.6.3]. In this paper, we will only need the following property, in which we examine the largest values of n we can get for a fixed value of $\ell(n)$.

Proposition 11 (Largest n for fixed $\ell(n)$). *Let $m \geq 2$. Then we have that*

1. *The largest value of n such that $\ell(n) = m$ is*

$$n = 2^m.$$

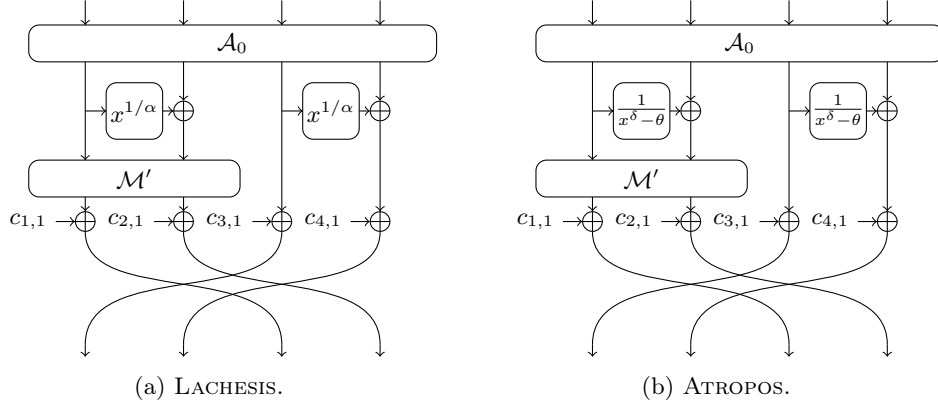


Fig. 7: First three layers of a MOIRAI with half-MDS layers.

2. The largest value of n such that n is odd and $\ell(n) = m$ is

$$n = 2^{m-1} + 1.$$

Proof. The first part of the proposition can be deduced from the fact that $\ell(2^m) = m$ and $\ell(n) \geq \lceil \log_2(n) \rceil$ (more generally, in an addition chain, $k_i \leq 2^i$). This can be found in [28, Section 4.6.3].

For the second part, we use induction. For $m = 2$, the only addition chains are $(1, 2, 3)$ and $(1, 2, 4)$, so the largest odd value for k_2 is 3. Suppose the proposition is true for all values less than a given $m \geq 3$. Then, consider an addition chain $(k_0, \dots, k_i, \dots, k_m)$ where k_m is odd and k_i is the largest odd number of the chain aside from k_m (which is always defined, as $k_0 = 1$). First, let us consider the case where $i \geq 2$. By induction, we have that $k_i \leq 2^{i-1} + 1$. Also, note that k_m must be a sum of an odd number and an even number, since it is odd. Therefore, k_m is upper bounded by the sum of k_i and the largest even number of the chain. Then, we consider two cases:

- If $i = m - 1$, both k_{m-1} and k_m are odd, and the largest even number of the chain is upper bounded by 2^{m-2} , by the first part of the proposition. Therefore,

$$k_m \leq 2^{m-2} + 1 + 2^{m-2} = 2^{m-1} + 1.$$

- If $i < m - 1$, the largest even number of the chain is k_{m-1} . We always have the upper bound $k_{m-1} \leq 2^{m-1-i} k_i$, and so

$$\begin{aligned} k_m &\leq k_i + k_{m-1} \\ &\leq 2^{i-1} + 1 + 2^{m-2} + 2^{m-1-i} \\ &< \sum_{a=0}^{m-2} 2^a \quad (\text{since } 2 \leq i < m) \\ &< 2^{m-1} + 1. \end{aligned}$$

This concludes the case where $i \geq 2$. We cannot have $i = 1$, as $k_1 = 2$. So, finally, let us consider the case where $i = 0$, i.e. all integers of the chain aside from k_0 are even. Again, k_m must be a sum between an odd integer and an even integer, but the only odd is 1. Therefore, we have that

$$k_m \leq k_{m-1} + 1 \leq 2^{m-1} + 1.$$

This concludes the induction. In fact, this bound is tight, and the addition chain $(1, 2, \dots, 2^{m-1}, 2^{m-1} + 1)$ yields the highest odd integer in an addition chain of size m . $\square$

C Additional proofs of Section 5

Proof of Proposition 5 for MDS layers. We will construct a polynomial model that solves the CICO- b problem such that it is already a Gröbner basis in grevlex order, following the Freelunch [7] method. Denote $\mathbf{M}$ as the matrix that represents the linear part of the MDS layers. Ensuring that the system has the correct shape is quite tricky: the inputs and outputs of the Sboxes need to be specific affine functions of the variables. We can do the following: first, we represent the input of the CICO- b problem as a sum of vectors

$$x_{1,1}\mathbf{v}_{1,0} + \dots + x_{b,1}\mathbf{v}_{b,0},$$

where $\mathbf{v}_{i,0} = (v_{i,0,1}, \dots, v_{i,0,b}, 0, \dots, 0) \in \mathbb{F}_p^{2b}$ is defined by a linear system of the form

$$\mathbf{M} \cdot \mathbf{v}_{i,0} = (\underbrace{0, \dots, 0}_i, \underbrace{1, 0, \dots, 0}_{b-i}, \underbrace{*, \dots, *}_b)$$

where 1 is at the i -th coordinate⁸, i.e. $\mathbf{M} \cdot \mathbf{v}_{i,0}$ is equal to 0 on left-side coordinates except at i where it is equal to 1.

Now, we proceed by induction. Let $1 \leq j \leq n_r$ and $1 \leq i \leq b$. Suppose that at the input of the j -th $\mathcal{S}$ layer, the i -th branch is a degree-1 polynomial, denoted $L_{i,j}$, in $\{x_{i,j-1}\} \cup \{x_{i',j'} : 1 \leq i' \leq b, 0 \leq j' < j-1\}$ (this is true for $j = 1$ due to the above paragraph). We now introduce variables $x_{i,j}$ for $1 \leq i \leq b$ so that the state after $\mathcal{S}$ will be of the form

$$x_{1,j}\mathbf{v}_{1,j} + \dots + x_{b,j}\mathbf{v}_{b,j} + \mathbf{w},$$

where

– $\mathbf{v}_{i,j} = (0, \dots, 0, v_{i,j,1}, \dots, v_{i,j,b}) \in \mathbb{F}_p^{2b}$ is defined by the same kind of linear system as earlier

$$\mathbf{M} \cdot \mathbf{v}_{i,j} = (\underbrace{0, \dots, 0}_i, \underbrace{1, 0, \dots, 0}_{b-i}, \underbrace{*, \dots, *}_b),$$

so as to satisfy the conditions for the next induction step.

⁸ Here, the index of the first coordinate is 1.

- $\mathbf{w}$ is the concatenation of two vectors of size b , $(*, \dots, *) \parallel (0, \dots, 0)$, where each $*$ represents a degree-1 polynomial in the variables of the set $\{x_{i',j'} : 1 \leq i' \leq b, 1 \leq j' \leq j-1\}$: these are the values of the branches that are not modified by the $\mathcal{S}$ layer.

For each $x_{i,j}$ variable, there will thus be a corresponding equation

$$(x_{i,j-1} + L_{i,j}(x_{i',j'} : 1 \leq i' \leq b, 0 \leq j' < j-1))^\beta = L'_{i,j}(x_{i',j'} : 1 \leq i' \leq b, 0 \leq j' \leq j),$$

where $L_{i,j}$ and $L'_{i,j}$ are polynomials of degree 1. The leading monomial of this equation is $x_{i,j-1}^\beta$, assuming that we use the grevlex order with a lexicographical order such that, if $j_1 < j_2$, then $x_{i_1,j_1} < x_{i_2,j_2}$ for all i_1, i_2, j_1, j_2 . By induction, we've fully defined all variables. The last b equations are affine and can be reduced so that their leading monomials are precisely $\{x_{i,n_r} : 1 \leq i \leq b\}$. In doing so, we've ensured that the set of leading monomials of the model with the appropriate grevlex order is exactly $\{x_{i,j}^\beta : 1 \leq i \leq b, 0 \leq j < n_r\} \cup \{x_{i,n_r} : 1 \leq i \leq b\}$. They are coprime leading monomials, therefore they define a Gröbner basis, and the ideal degree is the product of the degrees of the leading monomials (see [7, Prop.4]), and we can finally deduce Equation (3). $\square$

D Griffin

Here we give some results on Griffin [23]. We did not include it in the body of the paper because there is a lack of analysis (or even conjectures) of D_I outside of the CICO-1 case, which limits how far we can push its ALFOMs analysis. Griffin is a family of permutations that combines a modified Feistel scheme, called “Horst”, with a “Rescue-like” SPN. The current cryptanalytic works that tackle Griffin [7,6] provide a sound analysis of the polynomial systems, but only for the CICO-1 problem. Therefore, we will only consider the case $k = 1$. Note that the Griffin authors only listed instances where $p \approx 2^{256}$ and $\lambda = 128$ (see [23, Table 2]), so it is not too constraining to only study CICO-1.

Taking the results from the Freelunch paper, we have that

$$D_I(\alpha, b, n_r) = \begin{cases} (\alpha(2\alpha + 1))^{n_r-1} & \text{for } b \in \{3, 4\}, \\ 3(\alpha(2\alpha + 1))^{n_r-2} & \text{for } b = 8, \\ (6\alpha + 3)(\alpha(2\alpha + 1))^{n_r-3} & \text{for } b \geq 12. \end{cases}$$

For the number of constraints, we again refer to [12] – the original Griffin authors computed, for the number of R1CS constraints of a power- α map, $\lceil \log_2(\alpha) \rceil + hw(\alpha) - 1$, a tight upper bound for $\ell(\alpha)$.

$$\begin{aligned} C_{R1CS}(\alpha, b, n_r) &= (\ell(\alpha) + b - 2) \cdot 2n_r, \\ C_{AIR}(\alpha, b, n_r) &= \alpha \cdot b \cdot n_r. \end{aligned}$$

Now, we can deduce the ALFOMs for Griffin:

$$\eta_{R1CS}(\alpha, b, n_r) = \begin{cases} \frac{(n_r - 1) \log_2(\alpha(2\alpha + 1))}{2n_r(\ell(\alpha) + b - 2)} & \text{for } b \in \{3, 4\}, \\ \frac{\log_2(3) + (n_r - 2) \log_2(\alpha(2\alpha + 1))}{2n_r(\ell(\alpha) + 6)} & \text{for } b = 8, \\ \frac{\log_2(6\alpha + 3) + (n_r - 3) \log_2(\alpha(2\alpha + 1))}{2n_r(\ell(\alpha) + 10)} & \text{for } b \geq 12. \end{cases}$$

Here are some observations:

- Much like Poseidon, the ALFOMs of Griffin decreases with the number of branches. This becomes obvious when one considers that D_I does not increase with b , but the number of constraints does. It is as of yet unclear whether the ALFOMs would increase with k . However, Poseidon mitigates this issue with its many partial layers. For Griffin, it is clear that at constant $p \approx 2^{256}$ and security parameter $\lambda = 128$ (implicitly assuming that the security relies on the hardness of CICO-1), the best ALFOMs is achieved at the lowest number of branches, $b = 3$. This is doubly true when taking into account the number of skipped rounds, which increases with b .
- Asymptotically, as the number of rounds increases, we have the following:

$$\eta_{R1CS}(\alpha, b, n_r) \underset{n_r \rightarrow \infty}{\sim} \begin{cases} n_r \frac{\log_2(\alpha(2\alpha + 1))}{2(\ell(\alpha) + b - 2)} & \text{for } b \in \{3, 4\}, \\ n_r \frac{\log_2(\alpha(2\alpha + 1))}{2(\ell(\alpha) + 6)} & \text{for } b = 8, \\ n_r \frac{\log_2(\alpha(2\alpha + 1))}{2(\ell(\alpha) + 10)} & \text{for } b \geq 12. \end{cases}$$

However, keep in mind that the round skips here are particularly powerful (3 rounds out of 9 skipped for some instances where $b \geq 12$), so the asymptotic analysis will yield a looser approximation, compared to the other primitives in this paper.

E Experimental data on ATROPOS

Table 4: Experimental data of ATROPOS’s ideal degree for the model we describe in the proof of Prop. 9. A (✓) next to D_I means that the Bézout bound is reached, a (✗) means it is not reached.

(b, n_r, δ)	Bézout bound	D_I (MDS layers)	D_I (half-MDS internal layers)
(1, 1, 2)	3	3 (✓)	3 (✓)
(1, 1, 3)	4	4 (✓)	4 (✓)
(1, 2, 2)	9	9 (✓)	9 (✓)
(1, 2, 3)	16	16 (✓)	16 (✓)
(1, 3, 2)	27	27 (✓)	23 (✗)
(1, 3, 3)	64	64 (✓)	55 (✗)
(1, 4, 2)	81	81 (✓)	57 (✗)
(1, 4, 3)	256	256 (✓)	184 (✗)
(1, 5, 2)	243	243 (✓)	139 (✗)
(1, 6, 2)	729	729 (✓)	337 (✗)
(2, 1, 2)	9	9 (✓)	9 (✓)
(2, 1, 3)	16	16 (✓)	16 (✓)
(2, 2, 2)	81	81 (✓)	81 (✓)
(2, 2, 3)	256	256 (✓)	256 (✓)
(2, 3, 2)	729	729 (✓)	537 (✗)
(3, 1, 2)	27	27 (✓)	27 (✓)
(3, 1, 3)	64	64 (✓)	64 (✓)
(3, 2, 2)	729	729 (✓)	729 (✓)
(4, 1, 2)	81	81 (✓)	81 (✓)
(4, 1, 3)	256	256 (✓)	256 (✓)
(5, 1, 2)	243	243 (✓)	243 (✓)
(6, 1, 2)	729	729 (✓)	729 (✓)